DEVIXO SOLUTIONS

Data Science Internship Program

Exploratory Data Analysis

Sales Performance Dataset

TASK 01 REPORT

Student Name: Ateeb Mustapha

Internship Domain: Data Science

Task Number: Task 01

Task Title: Exploratory Data Analysis (EDA)

1. Objective

The objective of this task is to perform a complete Exploratory Data Analysis (EDA) on a real-world sales dataset using Python. The task covers the full analysis workflow: loading and understanding the data, cleaning it, analysing individual features, studying relationships between features, creating visualizations, and converting the findings into business insights and practical recommendations.

By the end of the task, the aim is to be able to explain what the data says about sales performance, which categories, regions and sales representatives drive revenue, and where the business can improve, with every statement supported by numbers from the analysis.

2. Problem Statement

A sales dataset of 1,000 transactions was selected from Kaggle. Each record describes one sale, including the sales representative, region, product category, quantity sold, unit cost, unit price, discount, customer type, payment method, sales channel and sale date. The raw data cannot be used for decisions directly, because it may contain missing values, duplicates, wrong data types or inconsistent entries, and because the patterns inside it are not visible from the table alone.

The problem addressed in this task is: how can the dataset be cleaned and explored to find out what drives revenue, which segments are strong or weak, whether discounts and quantities affect sales, and what practical actions a business could take based on that evidence?

3. Technologies Used

- **Python 3:** main programming language used for the whole analysis.
- **Pandas:** loading the CSV, cleaning, grouping and summarising data.
- **NumPy:** numerical operations used in the correlation analysis.
- **Matplotlib:** creating and customising charts.
- **Seaborn:** statistical visualizations such as heatmaps, box plots and violin plots.
- **Jupyter Notebook:** interactive environment used to write code and view output together.
- **Google Colab:** cloud notebook environment also used to run and test the code.
- **GitHub:** hosting the notebook, dataset, report and screenshots for submission.

4. Implementation Steps

1. **Dataset selection.** A sales dataset with 1,000 records and 14 columns was chosen from Kaggle, meeting the minimum size requirement.
2. **Data import and understanding.** The CSV was loaded with Pandas. The first and last rows, shape, column names and data types were inspected, and columns were split into numerical and categorical groups.

3. **Data cleaning.** Missing values were checked and filled (median for numbers, mode for text), duplicates were removed, date columns were converted to datetime, negative or invalid values were checked, and extra whitespace was stripped. The cleaned file was saved as sales_data_cleaned.csv.
4. **Feature analysis.** Minimum, maximum, mean and median were calculated for numerical features, category frequencies were reviewed, and outliers were detected with the IQR method.
5. **Correlation analysis.** A correlation matrix and heatmap were created to find strong, weak and important relationships between numerical features.
6. **Visualization.** Twelve charts were created: histogram, bar chart, count plot, pie chart, scatter plot, line chart, box plot, violin plot, heatmap, pair plot, distribution plot and area chart.
7. **Business analysis.** Total sales, averages, category, region and sales representative performance, and monthly trends were calculated and turned into business insights and recommendations.

5. Code Screenshots with Explanations

Data import and understanding

```
import pandas as pd
#part 1
df = pd.read_csv('sales_data.csv')
print(df.head())
print(df.tail())
print(df.columns)
print(df.dtypes)
numerical_cols = df.select_dtypes(include=["number"]).columns.tolist()
categorical_cols = df.select_dtypes(exclude=["number"]).columns.tolist()
print("Numerical Columns:", numerical_cols)
print("Categorical Columns:", categorical_cols)
summary = df.describe()
print(summary)
```

Figure: Data import and understanding

This code loads sales_data.csv with pd.read_csv() and prints the first and last rows, the column names and the data types. It then separates the columns into numerical and categorical lists using select_dtypes() and prints describe() to get basic statistics. This gives a first picture of the dataset before any changes are made.

Data cleaning

```
#part 2
print("missing values prior to cleaning: ")
print(df.isnull().sum())
print("total missing values:",df.isnull().sum().sum())
for col in numerical_cols:
    df[col] = df[col].fillna(df[col].median())
for col in categorical_cols:
    if df[col].isnull().any() and not df[col].mode().empty:
        df[col] = df[col].fillna(df[col].mode()[0])

print("Missing values after cleaning:")
print(df.isnull().sum())
print("Duplicate records before removal:", df.duplicated().sum())

df = df.drop_duplicates()

print("Duplicate records after removal:", df.duplicated().sum())

print("Data types before correction:")
print(df.dtypes)

if 'Order Date' in df.columns:
    df['Order Date'] = pd.to_datetime(
        df['Order Date'], errors='coerce'
    )

print("Data types after correction:")
print(df.dtypes)
for col in numerical_cols:
    print(f"Negative values in {col}:")
    print((df[col] < 0).sum())
```

```
for col in categorical_cols:
    print(f"Unique values in {col}:")
    print(df[col].value_counts(dropna=False).head(20))

for col in df.select_dtypes(include=["object"]).columns:
    df[col] = df[col].str.strip()

#Verifying the dataset after cleaning
print("First 5 rows:")
print(df.head())
print("Dataset shape (rows, columns):")
print(df.shape)
print("Missing values:")
print(df.isnull().sum())
print("Total missing values:", df.isnull().sum().sum())
print("Duplicate records:", df.duplicated().sum())
print("Final data types:")
print(df.dtypes)
print("Final summary statistics:")
print(df.describe(include="all"))
df.to_csv('sales_data_cleaned.csv', index=False)
print("Cleaned dataset saved as sales_data_cleaned.csv")
```

Figure: Data cleaning

This block checks missing values with isnull().sum() and fills numerical columns with the median and categorical columns with the mode. Duplicate rows are counted and removed with drop_duplicates(). Date columns are converted with pd.to_datetime(), negative values are checked in every numerical column, and whitespace is stripped from text columns. The final dataset is verified and saved as a cleaned CSV.

Feature analysis and outlier detection

```python
#part 3
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('sales_data_cleaned.csv')

numerical_cols = df.select_dtypes(include='number').columns.tolist()
categorical_cols = df.select_dtypes(include=['object', 'category']).columns.tolist()

print("Numerical Features")
print(df[numerical_cols].describe().T)

for col in numerical_cols:
    print(col, "Minimum:", df[col].min())
    print(col, "Maximum:", df[col].max())
    print(col, "Average:", df[col].mean())
    print(col, "Median:", df[col].median())

print("Categorical Features")

for col in categorical_cols:
    print(col, "Unique Categories:", df[col].nunique())
    print(df[col].value_counts().head(10))

for col in numerical_cols:
    plt.figure(figsize=(8, 4))
    sns.histplot(df[col].dropna(), kde=True)
    plt.title(f"Distribution of {col}")
    plt.tight_layout()
    plt.show()

for col in categorical_cols:
    if df[col].nunique() <= 20:
        plt.figure(figsize=(8, 4))
        df[col].value_counts().plot(kind='bar')
        plt.title(f"Distribution of {col}")
        plt.xticks(rotation=45)
        plt.tight_layout()
        plt.show()
```

```python
print("Outlier Analysis")

for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    print(col, "Outliers:", len(outliers))
    print("Lower Bound:", lower_bound)
    print("Upper Bound:", upper_bound)

if len(numerical_cols) > 1:
    plt.figure(figsize=(10, 6))
    sns.heatmap(df[numerical_cols].corr(), annot=True, cmap='coolwarm', fmt='.2f')
    plt.title("Correlation Between Numerical Features")
    plt.tight_layout()
    plt.show()


sales_cols = [
    col for col in df.columns
    if any(word in col.lower() for word in ['sales', 'revenue', 'profit', 'quantity'])
    and pd.api.types.is_numeric_dtype(df[col])
]

print("Important Business Features:", sales_cols)

for target in sales_cols:
    for category in categorical_cols:
        if df[category].nunique() <= 30:
            print(f"Average {target} by {category}")
            result = df.groupby(category)[target].mean()
            print(result.sort_values(ascending=False).head(10))


df[numerical_cols].describe().T.to_csv('numerical_feature_summary.csv')
```

Figure: Feature analysis and outlier detection

For each numerical column the code prints the minimum, maximum, average and median. For each categorical column it prints the number of unique values and the most frequent categories. The outlier section calculates Q1, Q3 and the IQR, then counts values outside the lower and upper bounds (Q1 - 1.5 x IQR and Q3 + 1.5 x IQR).

Correlation analysis

```
#part 4
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('sales_data_cleaned.csv')

numerical_cols = df.select_dtypes(include='number').columns.tolist()
corr = df[numerical_cols].corr()

print("Correlation Matrix:")
print(corr.round(2))

plt.figure(figsize=(10, 7))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f', vmin=-1, vmax=1)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()

strong_positive = []
strong_negative = []
weak_relationships = []

for i in range(len(corr.columns)):
    for j in range(i + 1, len(corr.columns)):
        col1 = corr.columns[i]
        col2 = corr.columns[j]
        value = corr.iloc[i, j]

        if pd.isna(value):
            continue

        if value >= 0.7:
            strong_positive.append((col1, col2, round(value, 2)))
        elif value <= -0.7:
            strong_negative.append((col1, col2, round(value, 2)))
        elif abs(value) < 0.3:
            weak_relationships.append((col1, col2, round(value, 2)))

print("\nStrong Positive Relationships:")
print(strong_positive if strong_positive else "None found")

print("\nStrong Negative Relationships:")
print(strong_negative if strong_negative else "None found")

print("\nWeak Relationships:")
print(weak_relationships if weak_relationships else "None found")

print("\nImportant Feature Relationships:")
for i in range(len(corr.columns)):
    for j in range(i + 1, len(corr.columns)):
        value = corr.iloc[i, j]
        if pd.notna(value) and abs(value) >= 0.5:
            print(f"{corr.columns[i]} and {corr.columns[j]}: {value:.2f}")
```

Figure: Correlation analysis

This code calculates the correlation matrix of all numerical columns, draws it as an annotated heatmap, and loops through every pair of columns to classify the relationship as strongly positive (0.7 or higher), strongly negative (-0.7 or lower) or weak (below 0.3 in absolute value).

Data visualization

```python
#part 5
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('sales_data_cleaned.csv')

sns.set_theme(style="whitegrid")
df.columns = df.columns.str.strip()

numerical_cols = df.select_dtypes(include='number').columns.tolist()
categorical_cols = df.select_dtypes(include=['object', 'category']).columns.tolist()

date_cols = []
for col in df.columns:
    if any(word in col.lower() for word in ['date', 'time']):
        converted = pd.to_datetime(df[col], errors='coerce')
        if converted.notna().sum() > 0:
            df[col] = converted
            date_cols.append(col)

categorical_cols = [col for col in categorical_cols if col not in date_cols]

sales_cols = [
    col for col in numerical_cols
    if any(word in col.lower() for word in ['sales', 'revenue', 'profit', 'quantity', 'amount'])
]

value_col = sales_cols[0] if sales_cols else (
    numerical_cols[0] if numerical_cols else None
)

category_col = next(
    (col for col in categorical_cols
     if any(word in col.lower() for word in ['category', 'product', 'region', 'segment', 'city'])),
    categorical_cols[0] if categorical_cols else None
)
```

```python
def finish_plot():
    plt.tight_layout()
    plt.show()

if value_col:
    plt.figure(figsize=(8, 5))
    sns.histplot(df[value_col].dropna(), bins=25, kde=True)
    plt.title(f'1. Distribution of {value_col}')
    plt.xlabel(value_col)
    plt.ylabel('Frequency')
    finish_plot()

if category_col and value_col:
    top = df.groupby(category_col, observed=True)[value_col].sum().nlargest(10)
    plt.figure(figsize=(9, 5))
    sns.barplot(x=top.values, y=top.index, color='steelblue')
    plt.title(f'2. Top Categories by Total {value_col}')
    plt.xlabel(f'Total {value_col}')
    plt.ylabel(category_col)
    finish_plot()

if category_col:
    counts = df[category_col].value_counts().head(15)
    plt.figure(figsize=(10, 5))
    sns.countplot(data=df[df[category_col].isin(counts.index)],
                  y=category_col, order=counts.index, color='cornflowerblue')
    plt.title(f'3. Record Count by {category_col}')
    plt.xlabel('Number of Records')
    plt.ylabel(category_col)
    finish_plot()

if category_col:
    pie_data = df[category_col].value_counts().head(6)
    if len(df[category_col].dropna().unique()) > 6:
        pie_data.loc['Other'] = len(df[category_col].dropna()) - pie_data.sum()
    plt.figure(figsize=(7, 7))
    plt.pie(pie_data, labels=pie_data.index, autopct='%1.1f%%',
            startangle=90)
    plt.title(f'4. Share of Records by {category_col}')
    plt.ylabel('')
```

```python
if len(numerical_cols) >= 2:
    x_col = next((c for c in numerical_cols if c.lower() in ['quantity', 'units', 'discount']), numerical_cols[0])
    y_col = next((c for c in sales_cols if c != x_col), numerical_cols[1])
    plt.figure(figsize=(8, 5))
    sns.scatterplot(data=df, x=x_col, y=y_col, alpha=0.65)
    plt.title(f'5. Relationship Between {x_col} and {y_col}')
    plt.xlabel(x_col)
    plt.ylabel(y_col)
    finish_plot()

if date_cols and value_col:
    date_col = date_cols[0]
    trend = df.dropna(subset=[date_col]).copy()
    trend = trend.set_index(date_col).resample('ME')[value_col].sum()
    if not trend.empty:
        plt.figure(figsize=(10, 5))
        plt.plot(trend.index, trend.values, marker='o')
        plt.title(f'6. Monthly {value_col} Trend')
        plt.xlabel('Month')
        plt.ylabel(f'Total {value_col}')
        plt.xticks(rotation=45)
        finish_plot()

if category_col and value_col:
    plt.figure(figsize=(10, 5))
    sns.boxplot(data=df, x=category_col, y=value_col)
    plt.title(f'7. Distribution of {value_col} Across {category_col}')
    plt.xlabel(category_col)
    plt.ylabel(value_col)
    plt.xticks(rotation=45)
    finish_plot()

if category_col and value_col:
    plt.figure(figsize=(10, 5))
    sns.violinplot(data=df, x=category_col, y=value_col, cut=0)
    plt.title(f'8. Density of {value_col} Across {category_col}')
    plt.xlabel(category_col)
    plt.ylabel(value_col)
    plt.xticks(rotation=45)
    finish_plot()
```

```python
if len(numerical_cols) >= 2:
    plt.figure(figsize=(10, 7))
    sns.heatmap(df[numerical_cols].corr(), annot=True, cmap='coolwarm',
                fmt='.2f', vmin=-1, vmax=1)
    plt.title('9. Numerical Feature Correlation Heatmap')
    finish_plot()

if len(numerical_cols) >= 2:
    pair_cols = numerical_cols[:4]
    pair_df = df[pair_cols].dropna()
    if len(pair_df) > 1:
        sns.pairplot(pair_df, diag_kind='hist', corner=True)
        plt.suptitle('10. Relationships Between Numerical Features', y=1.02)
        plt.show()

if value_col:
    plt.figure(figsize=(8, 5))
    sns.kdeplot(data=df, x=value_col, fill=True)
    plt.title(f'11. Density Distribution of {value_col}')
    plt.xlabel(value_col)
    plt.ylabel('Density')
    finish_plot()

if date_cols and value_col:
    date_col = date_cols[0]
    trend_df = df.dropna(subset=[date_col]).copy()
    trend_df = trend_df.set_index(date_col).resample('ME')[value_col].sum()
    if not trend_df.empty:
        plt.figure(figsize=(10, 5))
        plt.fill_between(trend_df.index, trend_df.values, alpha=0.4, color='teal')
        plt.plot(trend_df.index, trend_df.values, color='teal')
        plt.title(f'12. Monthly {value_col} Area Chart')
        plt.xlabel('Month')
        plt.ylabel(f'Total {value_col}')
        plt.xticks(rotation=45)
        finish_plot()
```

Figure: Data visualization

This script creates the 12 required charts using Matplotlib and Seaborn. Each chart is given a numbered title, axis labels and a suitable size. The code picks the sales column, a category column and the date column automatically, and groups monthly sales with resample() for the line and area charts.

Business metrics and insights

```python
import pandas as pd
import numpy as np

df = pd.read_csv('sales_data_cleaned.csv')
df.columns = df.columns.str.strip()

num_cols = df.select_dtypes(include='number').columns.tolist()
cat_cols = df.select_dtypes(include=['object', 'category']).columns.tolist()

sales_col = next((c for c in num_cols if 'sales' in c.lower() or 'revenue' in c.lower()), None)
profit_col = next((c for c in num_cols if 'profit' in c.lower()), None)
qty_col = next((c for c in num_cols if 'quantity' in c.lower() or 'units' in c.lower()), None)
discount_col = next((c for c in num_cols if 'discount' in c.lower()), None)

print("1. Dataset overview")
print("Rows and columns:", df.shape)
print("Date range and missing values:")
print(df.isnull().sum())

print("\n2. Numerical summary")
print(df[num_cols].describe().round(2))

print("\n3. Category performance")
for col in cat_cols:
    if df[col].nunique() <= 100:
        print(f"\nCategory: {col}")
        if sales_col:
            print("Highest sales:")
            print(df.groupby(col)[sales_col].sum().sort_values(ascending=False).head(5))
            print("Lowest sales:")
            print(df.groupby(col)[sales_col].sum().sort_values().head(5))
        if profit_col:
            print("Highest profit:")
            print(df.groupby(col)[profit_col].sum().sort_values(ascending=False).head(5))
            print("Lowest profit:")
            print(df.groupby(col)[profit_col].sum().sort_values().head(5))
```

```python
print("\n4. Sales and profit indicators")
if sales_col:
    print("Total sales:", round(df[sales_col].sum(), 2))
    print("Average sales:", round(df[sales_col].mean(), 2))
    print("Median sales:", round(df[sales_col].median(), 2))
    print("Highest transaction:", round(df[sales_col].max(), 2))
    print("Lowest transaction:", round(df[sales_col].min(), 2))

if profit_col:
    print("Total profit:", round(df[profit_col].sum(), 2))
    print("Average profit:", round(df[profit_col].mean(), 2))
    if sales_col:
        total_sales = df[sales_col].sum()
        print("Overall profit margin (%):",
              round(df[profit_col].sum() / total_sales * 100, 2)
              if total_sales != 0 else "Undefined")

if qty_col:
    print("Total quantity:", df[qty_col].sum())
    print("Average quantity:", round(df[qty_col].mean(), 2))

if discount_col:
    print("Average discount:", round(df[discount_col].mean(), 2))
    if profit_col:
        print("Discount-profit correlation:",
              round(df[discount_col].corr(df[profit_col]), 3))

print("\n5. Correlation analysis")
if len(num_cols) > 1:
    corr = df[num_cols].corr()
    pairs = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool)).stack()
    print(pairs.reindex(pairs.abs().sort_values(ascending=False).index).head(15).round(3))

print("\n6. Outlier analysis")
for col in num_cols:
    s = df[col].dropna()
    q1 = s.quantile(0.25)
    q3 = s.quantile(0.75)
    iqr = q3 - q1
    count = ((s < q1 - 1.5 * iqr) | (s > q3 + 1.5 * iqr)).sum()
```

```
print("\n7. Date trends")
date_cols = []
for col in df.columns:
    if 'date' in col.lower():
        dates = pd.to_datetime(df[col], errors='coerce')
        if dates.notna().sum() > 0:
            df[col] = dates
            date_cols.append(col)

if date_cols and sales_col:
    date_col = date_cols[0]
    monthly = df.dropna(subset=[date_col]).set_index(date_col)[sales_col].resample('ME').sum()
    print("Monthly sales:")
    print(monthly)
    print("Best month:", monthly.idxmax(), monthly.max()) if not monthly.empty else None
    print("Lowest month:", monthly.idxmin(), monthly.min()) if not monthly.empty else None

print("\n8. Customer analysis")
customer_col = next((c for c in cat_cols if 'customer' in c.lower()), None)
if customer_col:
    print("Unique customers:", df[customer_col].nunique())
    if sales_col:
        print("Top customers by sales:")
        print(df.groupby(customer_col)[sales_col].sum().sort_values(ascending=False).head(10))
    if profit_col:
        print("Top customers by profit:")
        print(df.groupby(customer_col)[profit_col].sum().sort_values(ascending=False).head(10))
```

Figure: Business metrics and insights

The final script calculates total sales, average and median sales, total quantity and average discount. It groups sales by every categorical column to find the highest and lowest performers, prints the strongest correlations, repeats the outlier check, and calculates monthly sales to find the best and worst months. These outputs were used to write the business insights.

6. Output / Result Screenshots with Explanations

Dataset overview

```
...     Product_ID   Sale_Date Sales_Rep Region  Sales_Amount  Quantity_Sold  \
0         1052  2023-02-03       Bob  North       5053.97             18
1         1093  2023-04-21       Bob   West       4384.02             17
2         1015  2023-09-21     David  South       4631.23             30
3         1072  2023-08-24       Bob  South       2167.94             39
4         1061  2023-03-24   Charlie   East       3750.20             13

  Product_Category  Unit_Cost  Unit_Price Customer_Type  Discount  \
0        Furniture     152.75      267.22     Returning      0.09
1        Furniture    3816.39     4209.44     Returning      0.11
2             Food     261.56      371.40     Returning      0.20
3         Clothing    4330.03     4467.75           New      0.02
4      Electronics     637.37      692.71           New      0.08

  Payment_Method Sales_Channel Region_and_Sales_Rep
0           Cash        Online            North-Bob
1           Cash        Retail             West-Bob
2  Bank Transfer        Retail           South-David
3    Credit Card        Retail             South-Bob
4    Credit Card        Online          East-Charlie
     Product_ID   Sale_Date Sales_Rep Region  Sales_Amount  Quantity_Sold  \
995        1010  2023-04-15   Charlie  North       4733.88              4
996        1067  2023-09-07       Bob  North       4716.36             37
997        1018  2023-04-27     David  South       7629.70             17
998        1100  2023-12-20     David   West       1629.47             39
999        1086  2023-08-16     Alice   East       4923.93             48

    Product_Category  Unit_Cost  Unit_Price Customer_Type  Discount  \
995             Food    4943.03     5442.15     Returning      0.29
996         Clothing    1754.32     1856.40           New      0.21
997         Clothing     355.72      438.27     Returning      0.06
998      Electronics    3685.03     3743.39           New      0.01
999             Food    2632.58     2926.68     Returning      0.14

    Payment_Method Sales_Channel Region_and_Sales_Rep
995           Cash        Online        North-Charlie
996  Bank Transfer        Retail            North-Bob
997  Bank Transfer        Online          South-David
998  Bank Transfer        Online           West-David
999           Cash        Online           East-Alice
```

```
Index(['Product_ID', 'Sale_Date', 'Sales_Rep', 'Region', 'Sales_Amount',
       'Quantity_Sold', 'Product_Category', 'Unit_Cost', 'Unit_Price',
       'Customer_Type', 'Discount', 'Payment_Method', 'Sales_Channel',
       'Region_and_Sales_Rep'],
      dtype='object')
Product_ID              int64
Sale_Date              object
Sales_Rep              object
Region                 object
Sales_Amount          float64
Quantity_Sold           int64
Product_Category       object
Unit_Cost             float64
Unit_Price            float64
Customer_Type          object
Discount              float64
Payment_Method         object
Sales_Channel          object
Region_and_Sales_Rep   object
dtype: object
Numerical Columns: ['Product_ID', 'Sales_Amount', 'Quantity_Sold', 'Unit_Cost', 'Unit_Price', 'Discount']
Categorical Columns: ['Sale_Date', 'Sales_Rep', 'Region', 'Product_Category', 'Customer_Type', 'Payment_Method', 'Sales_Channel', 'Region_and_Sales_Rep']
```

Figure: Dataset overview

The dataset contains 1,000 rows and 14 columns. Six columns are numerical (Product_ID, Sales_Amount, Quantity_Sold, Unit_Cost, Unit_Price, Discount) and eight are categorical, including Sale_Date, Sales_Rep, Region, Product_Category, Customer_Type, Payment_Method, Sales_Channel and Region_and_Sales_Rep. This confirms the dataset meets the minimum requirement of 1,000 records with multiple features.

Basic statistical summary

	Product_ID	Sales_Amount	Quantity_Sold	Unit_Cost	Unit_Price
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	1050.128000	5019.265230	25.355000	2475.304550	2728.440120
std	29.573505	2846.790126	14.159006	1417.872546	1419.399839
min	1001.000000	100.120000	1.000000	60.280000	167.120000
25%	1024.000000	2550.297500	13.000000	1238.380000	1509.085000
50%	1051.000000	5019.300000	25.000000	2467.235000	2696.400000
75%	1075.000000	7507.445000	38.000000	3702.865000	3957.970000
max	1100.000000	9989.040000	49.000000	4995.300000	5442.150000

	Discount
count	1000.00000
mean	0.15239
std	0.08720
min	0.00000
25%	0.08000
50%	0.15000
75%	0.23000
max	0.30000

Figure: Basic statistical summary

Sales_Amount ranges from 100.12 to 9,989.04 with a mean of 5,019.27 and a median of 5,019.30. Because the mean and median are almost identical, the distribution is symmetric. Quantity_Sold ranges from 1 to 49 units (mean 25.36), Unit_Price ranges from 167.12 to 5,442.15, and Discount ranges from 0 to 0.30 with an average of 0.15 (15%).

Data cleaning verification

```
    missing values prior to cleaning:
    Product_ID              0
...  Sale_Date               0
    Sales_Rep               0
    Region                  0
    Sales_Amount            0
    Quantity_Sold           0
    Product_Category        0
    Unit_Cost               0
    Unit_Price              0
    Customer_Type           0
    Discount                0
    Payment_Method          0
    Sales_Channel           0
    Region_and_Sales_Rep    0
    dtype: int64
    total missing values: 0
    Missing values after cleaning:
    Product_ID              0
    Sale_Date               0
    Sales_Rep               0
    Region                  0
    Sales_Amount            0
    Quantity_Sold           0
    Product_Category        0
    Unit_Cost               0
    Unit_Price              0
    Customer_Type           0
    Discount                0
    Payment_Method          0
    Sales_Channel           0
    Region_and_Sales_Rep    0
    dtype: int64
    Duplicate records before removal: 0
    Duplicate records after removal: 0
    Data types before correction:
    Product_ID               int64
    Sale_Date               object
    Sales_Rep               object
    Region                  object
    Sales_Amount           float64
    Quantity_Sold            int64
    Product_Category        object
    Unit Cost              float64
```

```
Unit_Price              float64
Customer_Type            object
Discount                float64
Payment_Method           object
Sales_Channel            object
Region_and_Sales_Rep     object
dtype: object
Data types after correction:
Product_ID                int64
Sale_Date                object
Sales_Rep                object
Region                   object
Sales_Amount            float64
Quantity_Sold             int64
Product_Category         object
Unit_Cost               float64
Unit_Price              float64
Customer_Type            object
Discount                float64
Payment_Method           object
Sales_Channel            object
Region_and_Sales_Rep     object
dtype: object
Negative values in Product_ID:
0
Negative values in Sales_Amount:
0
Negative values in Quantity_Sold:
0
Negative values in Unit_Cost:
0
Negative values in Unit_Price:
0
Negative values in Discount:
0
Unique values in Sale_Date:
Sale_Date
2023-11-14    8
2023-10-20    8
2023-01-12    7
2023-10-16    7
2023-08-14    7
2023-10-15    7
2023-11-24    7
```

```
2023-02-08    6
2023-08-15    6
2023-02-28    6
2023-01-04    6
2023-01-29    6
2023-12-15    6
2023-04-14    6
2023-08-03    6
2023-10-12    6
2023-01-25    6
2023-01-17    6
2023-02-09    6
2023-12-22    6
Name: count, dtype: int64
Unique values in Sales_Rep:
Sales_Rep
David      222
Eve        209
Bob        208
Alice      192
Charlie    169
Name: count, dtype: int64
Unique values in Region:
Region
North    267
East     263
West     244
South    226
Name: count, dtype: int64
Unique values in Product_Category:
Product_Category
Clothing       268
Furniture      260
Electronics    246
Food           226
Name: count, dtype: int64
Unique values in Customer_Type:
Customer_Type
New          504
Returning    496
Name: count, dtype: int64
Unique values in Payment_Method:
Payment_Method
Credit Card      345
```

```
Bank Transfer    342
Cash             313
Name: count, dtype: int64
Unique values in Sales_Channel:
Sales_Channel
Retail    512
Online    488
Name: count, dtype: int64
Unique values in Region_and_Sales_Rep:
Region_and_Sales_Rep
North-Eve        64
East-Bob         60
East-David       59
South-David      56
North-David      56
East-Eve         55
North-Charlie    55
West-Bob         54
East-Alice       52
West-David       51
South-Bob        50
North-Alice      48
West-Alice       47
West-Eve         47
South-Alice      45
West-Charlie     45
North-Bob        44
South-Eve        43
East-Charlie     37
South-Charlie    32
Name: count, dtype: int64
First 5 rows:
   Product_ID   Sale_Date Sales_Rep Region  Sales_Amount  Quantity_Sold  \
0        1052  2023-02-03       Bob  North       5053.97             18
1        1093  2023-04-21       Bob   West       4384.02             17
2        1015  2023-09-21     David  South       4631.23             30
3        1072  2023-08-24       Bob  South       2167.94             39
4        1061  2023-03-24   Charlie   East       3750.20             13

  Product_Category  Unit_Cost  Unit_Price Customer_Type  Discount  \
0        Furniture     152.75      267.22     Returning      0.09
1        Furniture    3816.39     4209.44     Returning      0.11
2             Food     261.56      371.40     Returning      0.20
3         Clothing    4330.03     4467.75           New      0.02
```

```
    4      Electronics     637.37      692.71          New      0.08

...   Payment_Method Sales_Channel Region_and_Sales_Rep
    0           Cash        Online            North-Bob
    1           Cash        Retail             West-Bob
    2  Bank Transfer        Retail          South-David
    3    Credit Card        Retail            South-Bob
    4    Credit Card        Online         East-Charlie
    Dataset shape (rows, columns):
    (1000, 14)
    Missing values:
    Product_ID              0
    Sale_Date               0
    Sales_Rep               0
    Region                  0
    Sales_Amount            0
    Quantity_Sold           0
    Product_Category        0
    Unit_Cost               0
    Unit_Price              0
    Customer_Type           0
    Discount                0
    Payment_Method          0
    Sales_Channel           0
    Region_and_Sales_Rep    0
    dtype: int64
    Total missing values: 0
    Duplicate records: 0
    Final data types:
    Product_ID               int64
    Sale_Date               object
    Sales_Rep               object
    Region                  object
    Sales_Amount           float64
    Quantity_Sold            int64
    Product_Category        object
    Unit_Cost              float64
    Unit_Price             float64
    Customer_Type           object
    Discount               float64
    Payment_Method          object
    Sales_Channel           object
    Region_and_Sales_Rep    object
    dtype: object
```

```
Final summary statistics:
         Product_ID   Sale_Date Sales_Rep Region  Sales_Amount  Quantity_Sold  \
... count   1000.000000        1000      1000   1000   1000.000000    1000.000000
    unique          NaN         340         5      4           NaN            NaN
    top             NaN  2023-11-14     David  North           NaN            NaN
    freq            NaN           8       222    267           NaN            NaN
    mean    1050.128000         NaN       NaN    NaN   5019.265230      25.355000
    std       29.573505         NaN       NaN    NaN   2846.790126      14.159006
    min     1001.000000         NaN       NaN    NaN    100.120000       1.000000
    25%     1024.000000         NaN       NaN    NaN   2550.297500      13.000000
    50%     1051.000000         NaN       NaN    NaN   5019.300000      25.000000
    75%     1075.000000         NaN       NaN    NaN   7507.445000      38.000000
    max     1100.000000         NaN       NaN    NaN   9989.040000      49.000000

           Product_Category    Unit_Cost   Unit_Price Customer_Type    Discount  \
    count              1000  1000.000000  1000.000000          1000  1000.00000
    unique                4          NaN          NaN             2         NaN
    top            Clothing          NaN          NaN           New         NaN
    freq                268          NaN          NaN           504         NaN
    mean                NaN  2475.304550  2728.440120           NaN     0.15239
    std                 NaN  1417.872546  1419.399839           NaN     0.08720
    min                 NaN    60.280000   167.120000           NaN     0.00000
    25%                 NaN  1238.380000  1509.085000           NaN     0.08000
    50%                 NaN  2467.235000  2696.400000           NaN     0.15000
    75%                 NaN  3702.865000  3957.970000           NaN     0.23000
    max                 NaN  4995.300000  5442.150000           NaN     0.30000

           Payment_Method Sales_Channel Region_and_Sales_Rep
    count            1000          1000                1000
    unique              3             2                  20
    top       Credit Card        Retail           North-Eve
    freq              345           512                  64
    mean              NaN           NaN                 NaN
    std               NaN           NaN                 NaN
    min               NaN           NaN                 NaN
    25%               NaN           NaN                 NaN
    50%               NaN           NaN                 NaN
    75%               NaN           NaN                 NaN
    max               NaN           NaN                 NaN
    Cleaned dataset saved as sales_data_cleaned.csv
```

Figure: Data cleaning verification

After cleaning, the dataset has 1,000 rows and 14 columns with zero missing values and zero duplicate records. No negative values were found, since the smallest Sales_Amount is 100.12, the smallest

Quantity_Sold is 1 and the smallest Discount is 0. The dataset is therefore consistent and ready for analysis.

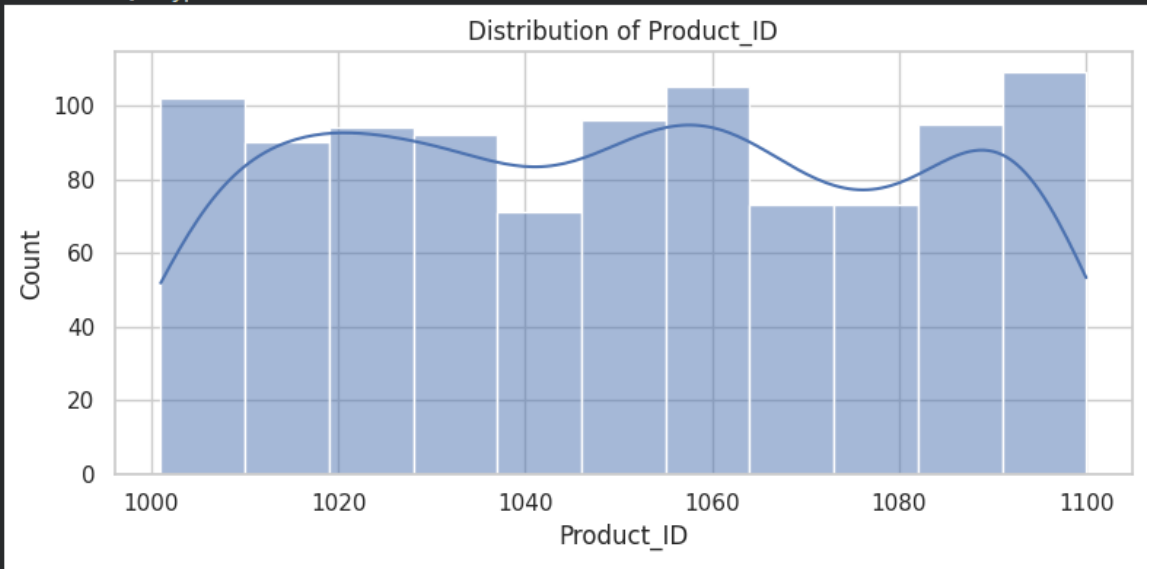
Categorical feature distribution

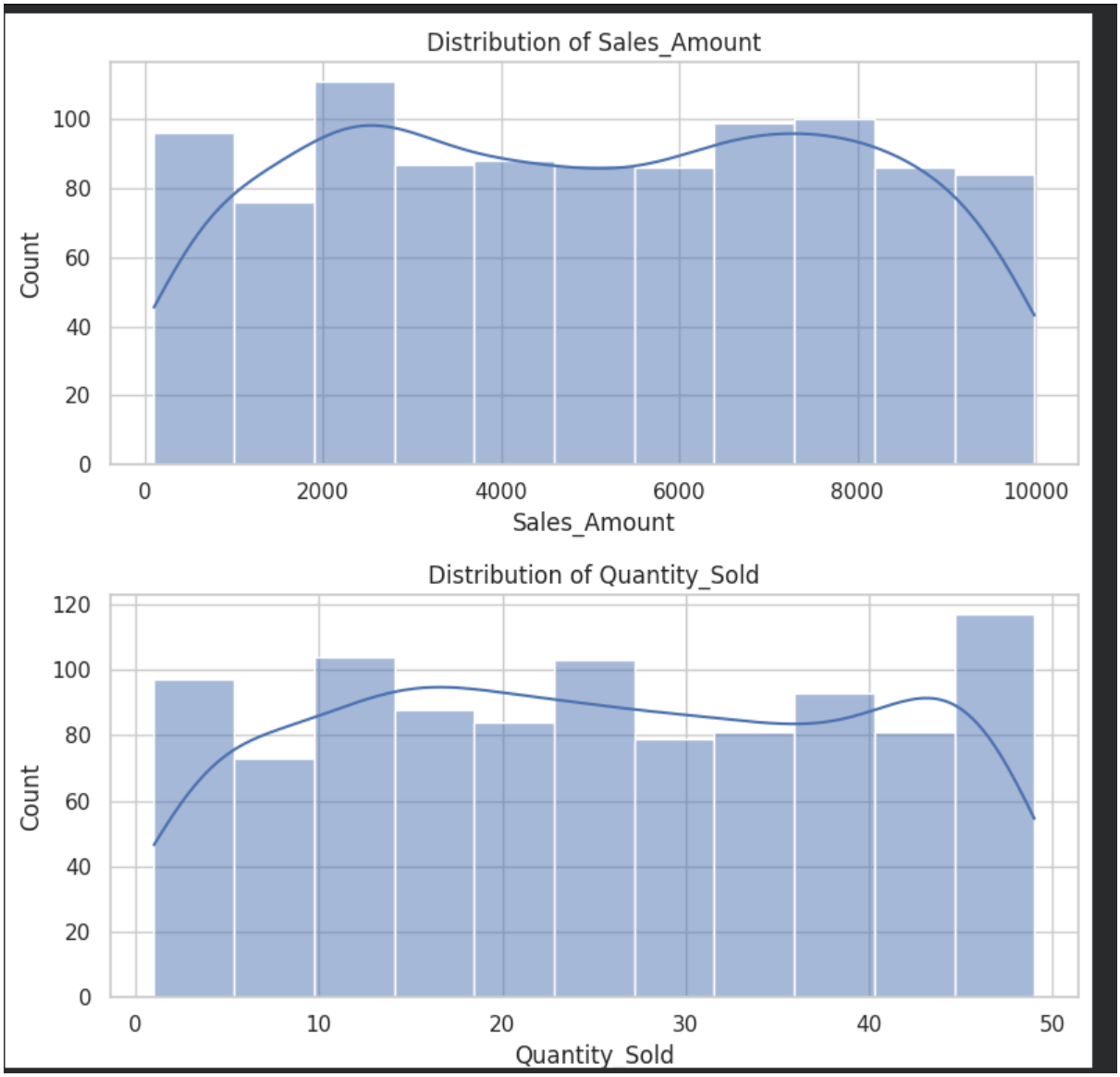
```
    Numerical Features
                    count         mean          std      min         25%         50%  \
...  Product_ID     1000.0  1050.12800    29.573505  1001.00  1024.0000  1051.000
     Sales_Amount   1000.0  5019.26523  2846.790126   100.12  2550.2975  5019.300
     Quantity_Sold  1000.0    25.35500    14.159006     1.00    13.0000    25.000
     Unit_Cost      1000.0  2475.30455  1417.872546    60.28  1238.3800  2467.235
     Unit_Price     1000.0  2728.44012  1419.399839   167.12  1509.0850  2696.400
     Discount       1000.0     0.15239     0.087200     0.00     0.0800     0.150

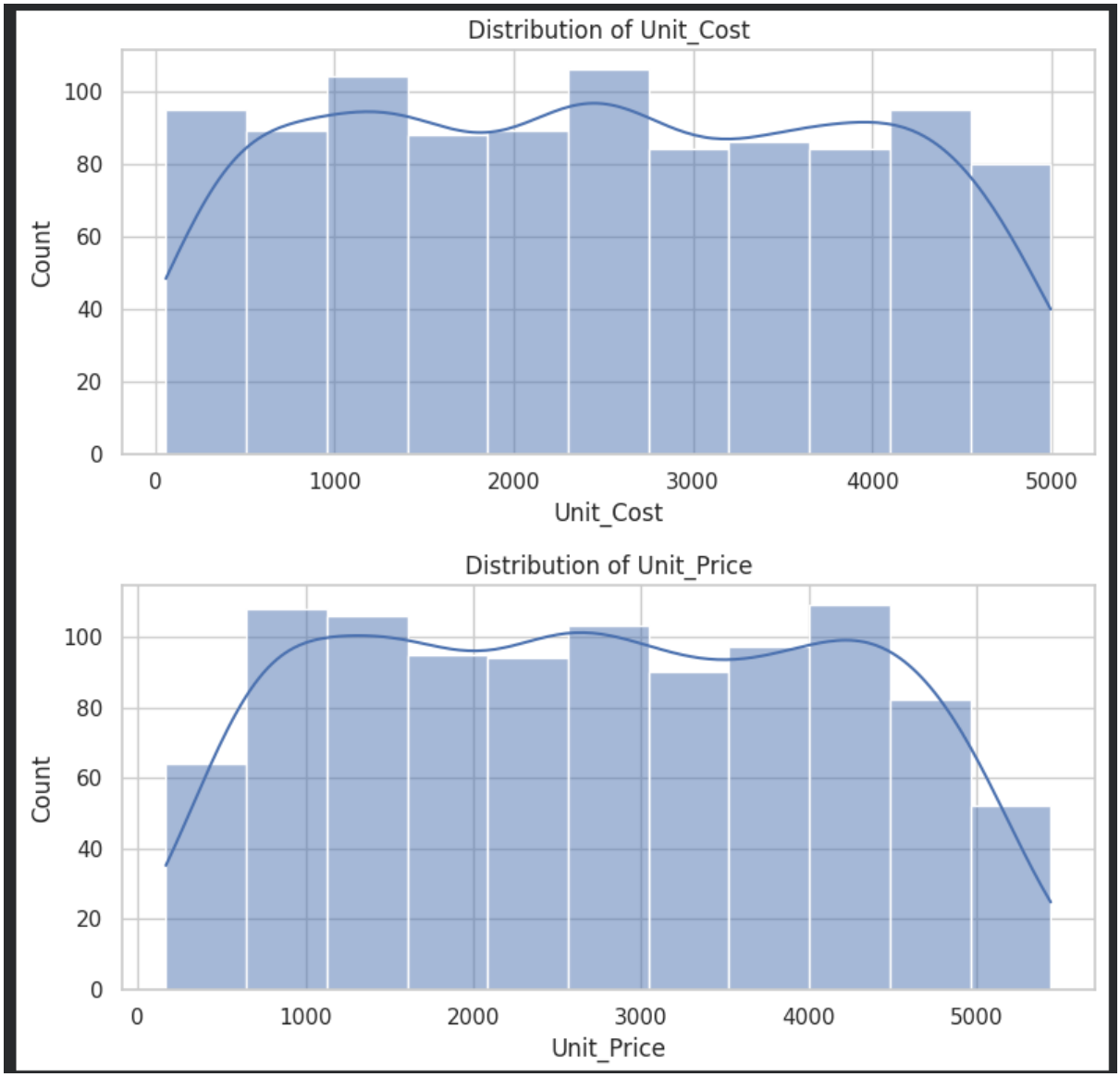
                        75%      max
     Product_ID     1075.000  1100.00
     Sales_Amount   7507.445  9989.04
     Quantity_Sold    38.000    49.00
     Unit_Cost      3702.865  4995.30
     Unit_Price     3957.970  5442.15
     Discount          0.230     0.30
     Product_ID Minimum: 1001
     Product_ID Maximum: 1100
     Product_ID Average: 1050.128
     Product_ID Median: 1051.0
     Sales_Amount Minimum: 100.12
     Sales_Amount Maximum: 9989.04
     Sales_Amount Average: 5019.265229999999
     Sales_Amount Median: 5019.3
     Quantity_Sold Minimum: 1
     Quantity_Sold Maximum: 49
     Quantity_Sold Average: 25.355
     Quantity_Sold Median: 25.0
     Unit_Cost Minimum: 60.28
     Unit_Cost Maximum: 4995.3
     Unit_Cost Average: 2475.3045500000003
     Unit_Cost Median: 2467.2349999999997
     Unit_Price Minimum: 167.12
     Unit_Price Maximum: 5442.15
     Unit_Price Average: 2728.44012
     Unit_Price Median: 2696.4
     Discount Minimum: 0.0
     Discount Maximum: 0.3
     Discount Average: 0.15239
     Discount Median: 0.15
     Categorical Features
     Sale_Date Unique Categories: 340
     Sale_Date
     2023-11-14    8
```

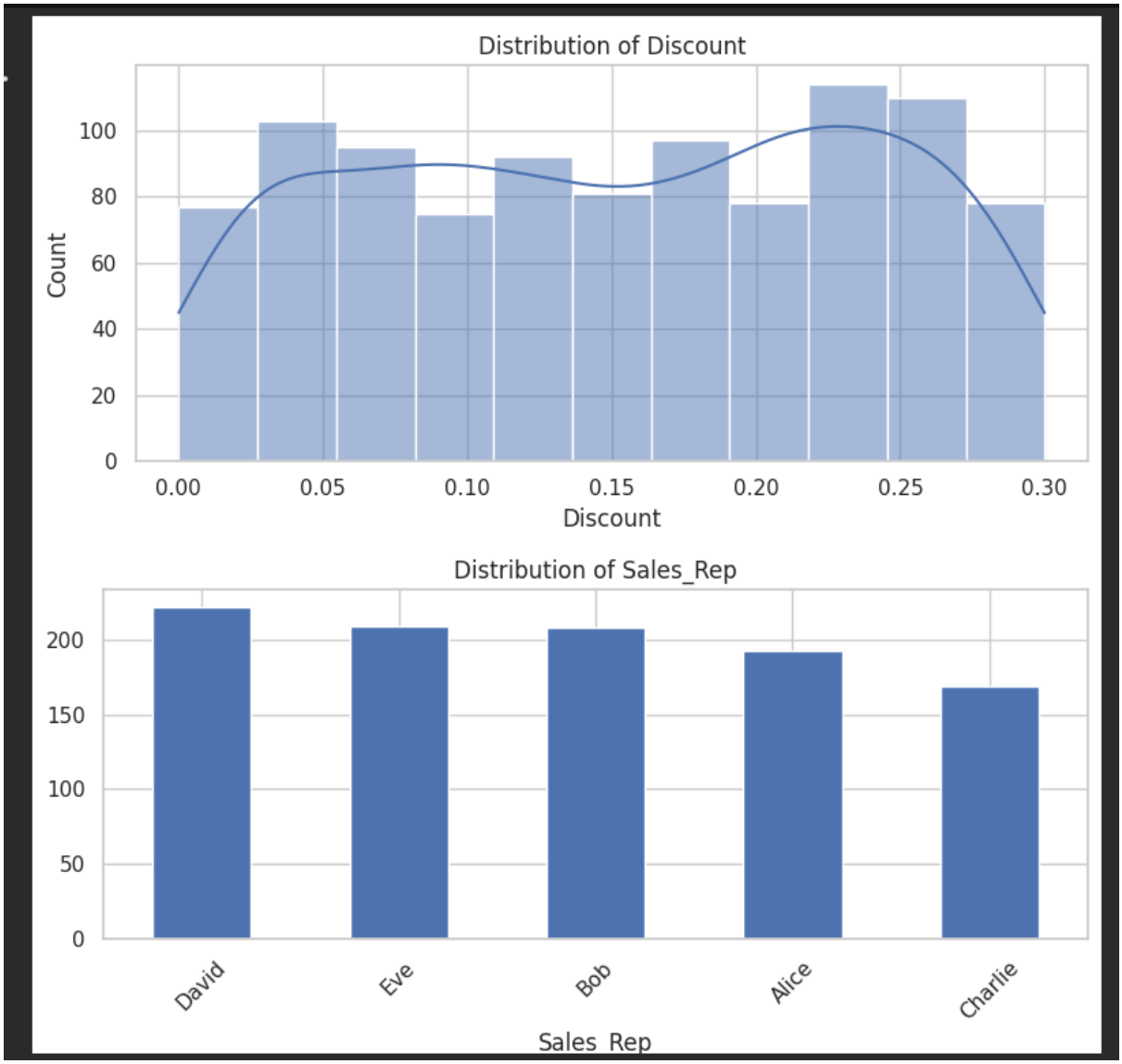
```
2023-10-20    8
2023-01-12    7
2023-10-16    7
2023-08-14    7
2023-10-15    7
2023-11-24    7
2023-02-08    6
2023-08-15    6
2023-02-28    6
Name: count, dtype: int64
Sales_Rep Unique Categories: 5
Sales_Rep
David      222
Eve        209
Bob        208
Alice      192
Charlie    169
Name: count, dtype: int64
Region Unique Categories: 4
Region
North    267
East     263
West     244
South    226
Name: count, dtype: int64
Product_Category Unique Categories: 4
Product_Category
Clothing       268
Furniture      260
Electronics    246
Food           226
Name: count, dtype: int64
Customer_Type Unique Categories: 2
Customer_Type
New          504
Returning    496
Name: count, dtype: int64
Payment_Method Unique Categories: 3
Payment_Method
Credit Card      345
Bank Transfer    342
Cash             313
Name: count, dtype: int64
Sales_Channel Unique Categories: 2
```

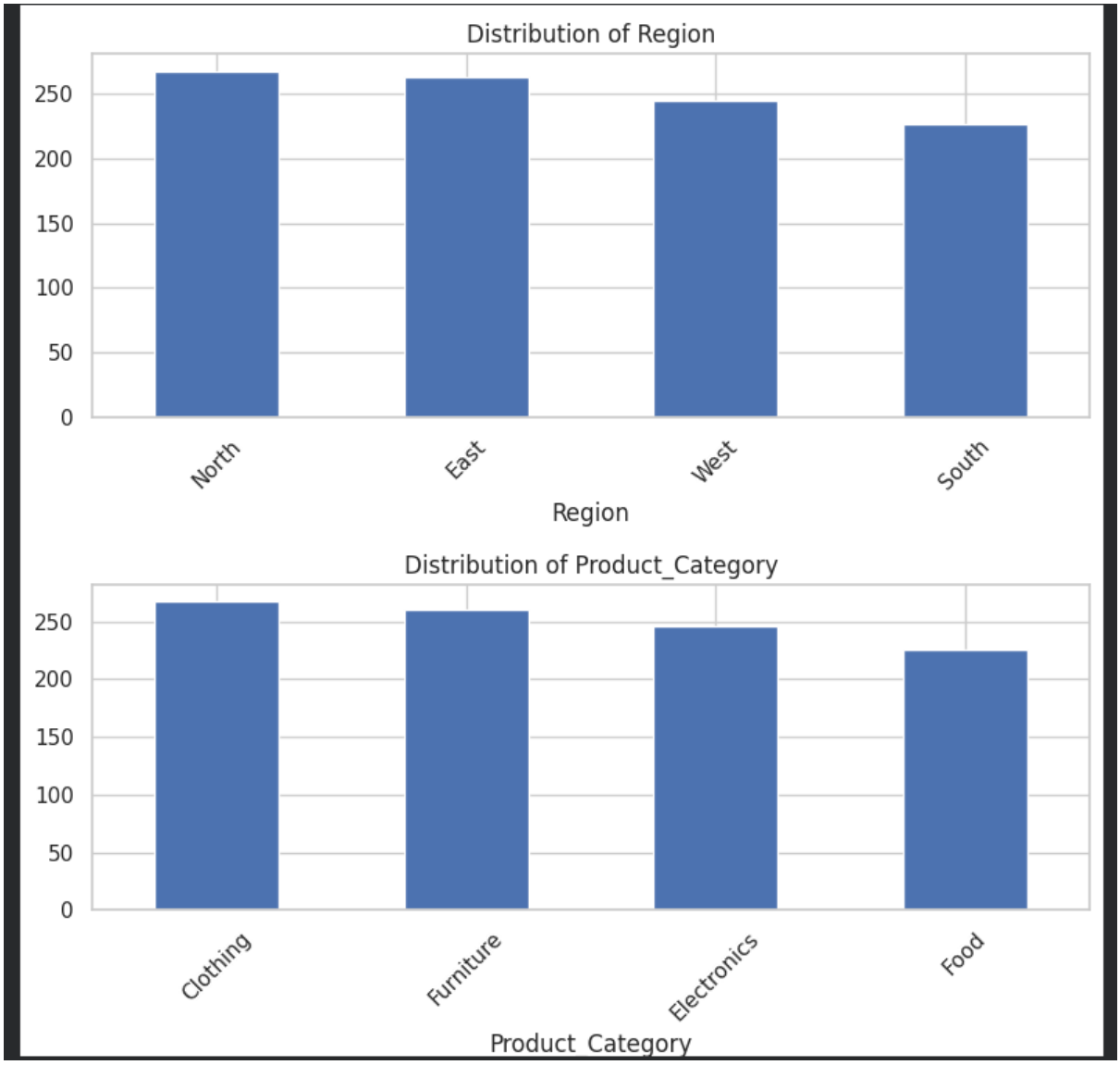
```
Sales_Channel
Retail    512
Online    488
Name: count, dtype: int64
Region_and_Sales_Rep Unique Categories: 20
Region_and_Sales_Rep
North-Eve        64
East-Bob         60
East-David       59
South-David      56
North-David      56
East-Eve         55
North-Charlie    55
West-Bob         54
East-Alice       52
West-David       51
Name: count, dtype: int64
```

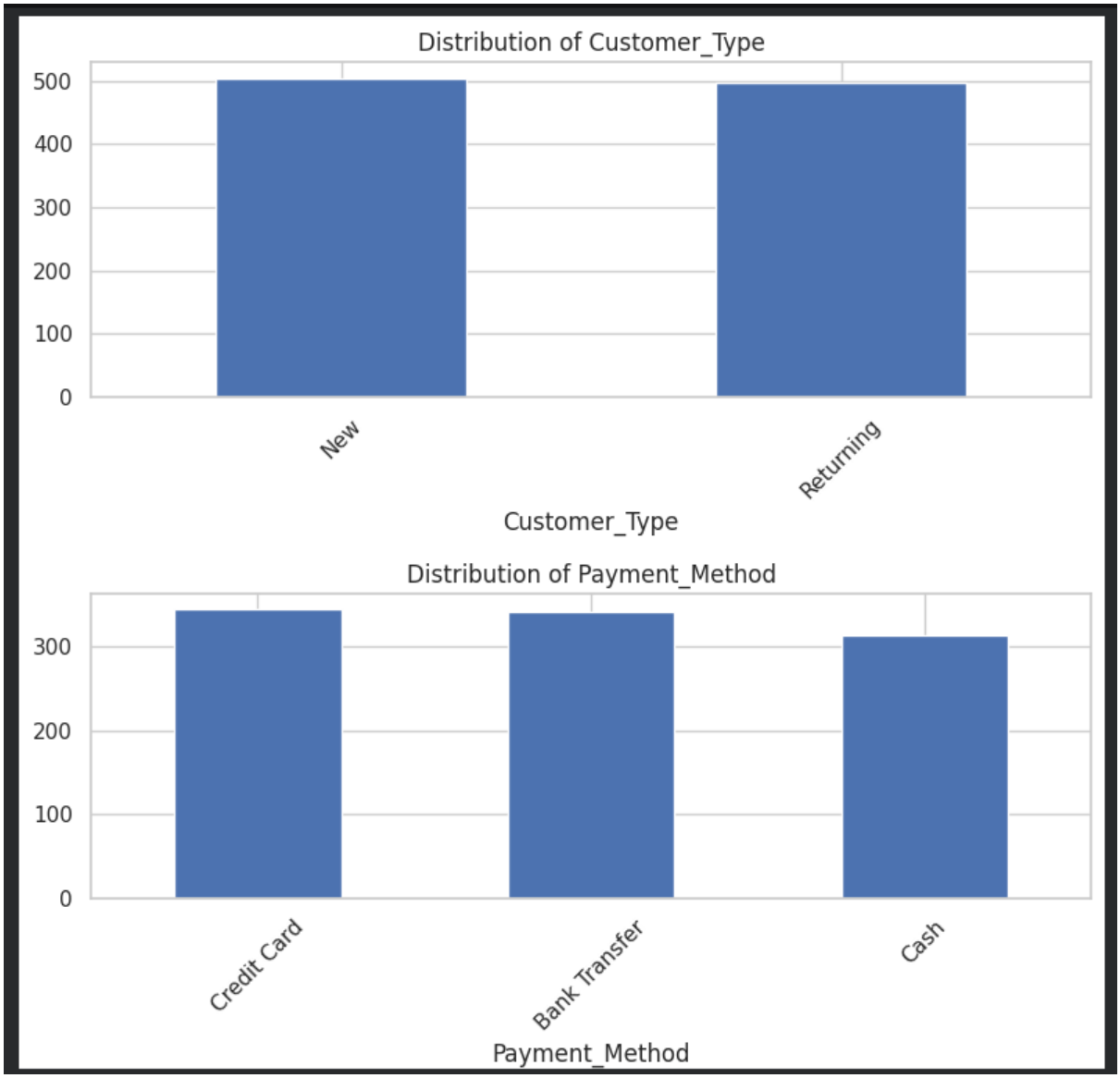



Distribution of Sales_Amount
Count
Sales_Amount
0
20
40
60
80
100
0
2000
4000
6000
8000
10000
Distribution of Quantity_Sold
Count
Quantity_Sold
0
20
40
60
80
100
120
0
10
20
30
40
50


Distribution of Unit_Cost
Count
Unit_Cost
0
20
40
60
80
100
0
1000
2000
3000
4000
5000
Distribution of Unit_Price
Count
Unit_Price
0
20
40
60
80
100
0
1000
2000
3000
4000
5000


Distribution of Discount
Count
Discount
0
20
40
60
80
100
0.00
0.05
0.10
0.15
0.20
0.25
0.30
Distribution of Sales_Rep
Sales_Rep
0
50
100
150
200
David
Eve
Bob
Alice
Charlie


Distribution of Region
250
200
150
100
50
0
North
East
West
South
Region
Distribution of Product_Category
250
200
150
100
50
0
Clothing
Furniture
Electronics
Food
Product_Category


Distribution of Customer_Type
500
400
300
200
100
0
New
Returning
Customer_Type
Distribution of Payment_Method
300
200
100
0
Credit Card
Bank Transfer
Cash
Payment_Method

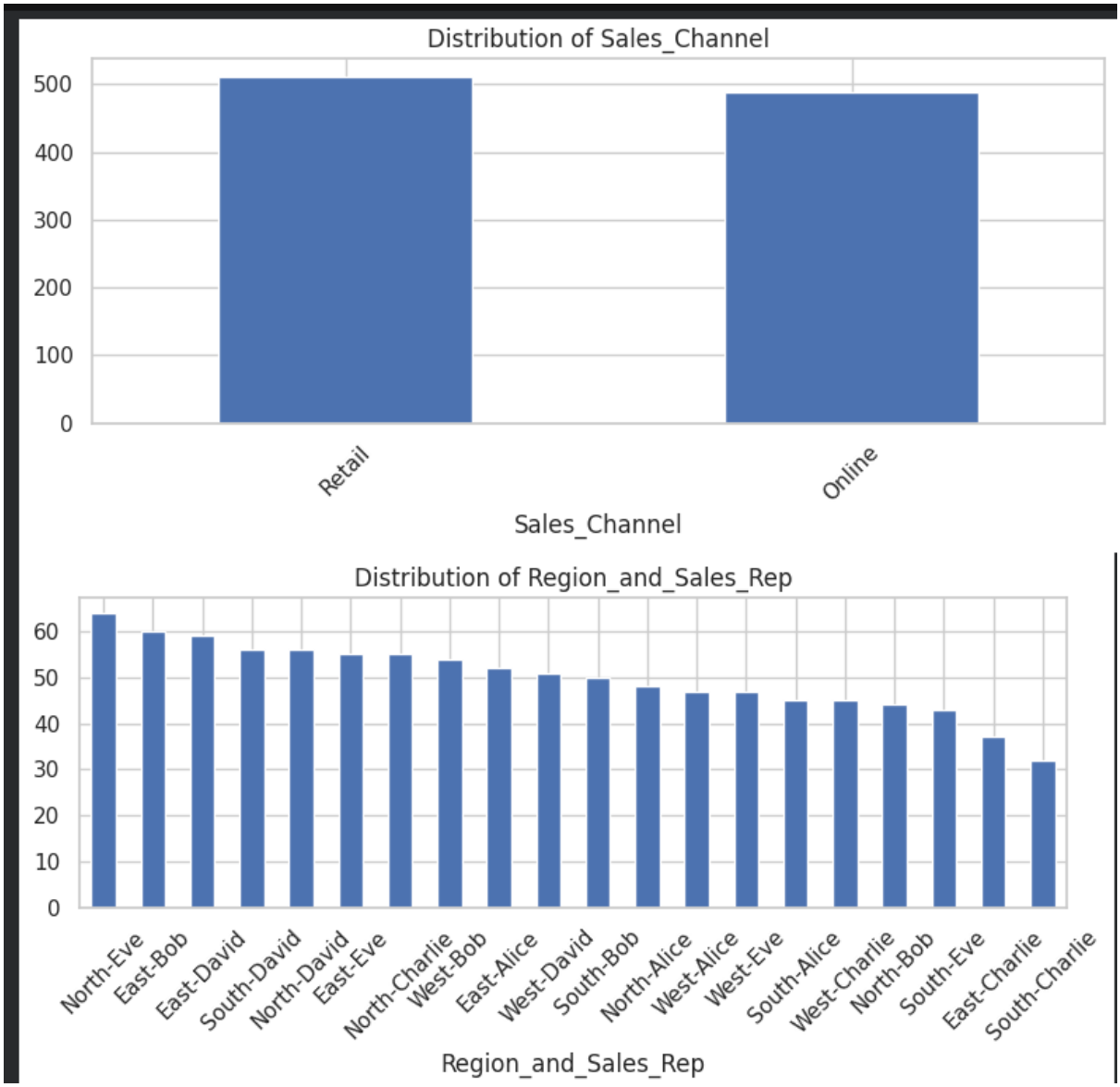
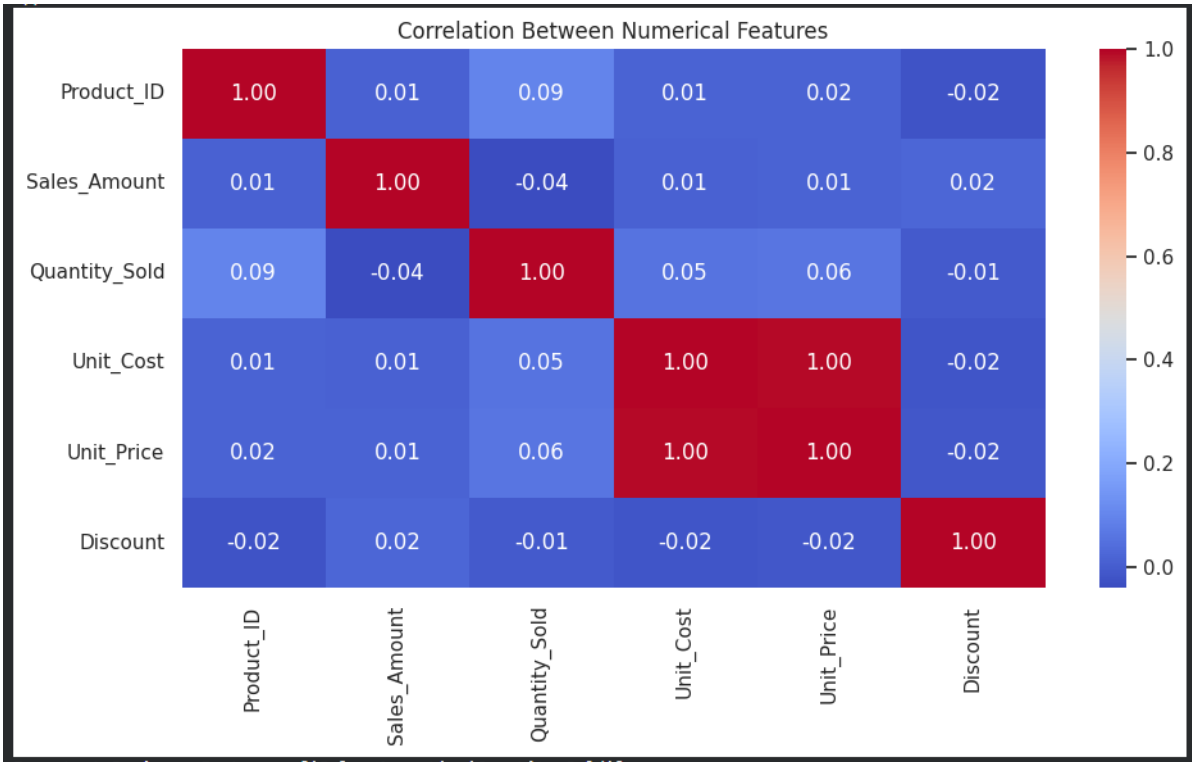


Figure: Categorical feature distribution

Clothing is the most frequent product category (268 records), followed by Furniture (260), Electronics (246) and Food (226). North has the most transactions (267) and South the fewest (226). David handled the most sales (222) and Charlie the fewest (169). Customers are split almost evenly between New (504) and Returning (496), and Retail (512) is slightly ahead of Online (488). Credit Card is the most used payment method (345).

Outlier analysis

```
Outlier Analysis
Product_ID Outliers: 0
Lower Bound: 947.5
Upper Bound: 1151.5
Sales_Amount Outliers: 0
Lower Bound: -4885.423750000002
Upper Bound: 14943.166250000002
Quantity_Sold Outliers: 0
Lower Bound: -24.5
Upper Bound: 75.5
Unit_Cost Outliers: 0
Lower Bound: -2458.3475000000008
Upper Bound: 7399.592500000001
Unit_Price Outliers: 0
Lower Bound: -2164.2425000000003
Upper Bound: 7631.297500000001
Discount Outliers: 0
Lower Bound: -0.14500000000000002
Upper Bound: 0.45500000000000007
```




```
Important Business Features: ['Sales_Amount', 'Quantity_Sold']
Average Sales_Amount by Sales_Rep
Sales_Rep
Bob        5197.070337
David      5142.961081
Charlie    5093.559053
Alice      5028.863385
Eve        4642.028660
```

```
Name: Sales_Amount, dtype: float64
Average Sales_Amount by Region
Region
North    5129.634869
South    5107.304690
West     5063.971025
East     4790.087186
Name: Sales_Amount, dtype: float64
Average Sales_Amount by Product_Category
Product_Category
Food           5317.582035
Electronics    5054.876585
Clothing       4901.023731
Furniture      4848.144962
Name: Sales_Amount, dtype: float64
Average Sales_Amount by Customer_Type
Customer_Type
Returning    5066.546230
New          4972.734722
Name: Sales_Amount, dtype: float64
Average Sales_Amount by Payment_Method
Payment_Method
Credit Card      5094.387014
Bank Transfer    5021.529737
Cash             4933.988946
Name: Sales_Amount, dtype: float64
Average Sales_Amount by Sales_Channel
Sales_Channel
Online    5038.594119
Retail    5000.842383
Name: Sales_Amount, dtype: float64
Average Sales_Amount by Region_and_Sales_Rep
Region_and_Sales_Rep
South-David      5561.017679
North-Charlie    5450.709273
West-Charlie     5326.863333
South-Alice      5320.323111
North-Bob        5306.018409
West-Bob         5304.616296
North-Alice      5200.983750
East-Bob         5164.601833
West-David       5160.148824
North-David      5045.344107
Name: Sales Amount, dtype: float64
```

```
Average Quantity_Sold by Sales_Rep
Sales_Rep
David      27.216216
Eve        25.296651
Alice      25.166667
Charlie    24.952663
Bob        23.927885
Name: Quantity_Sold, dtype: float64
Average Quantity_Sold by Region
Region
West     26.581967
South    25.699115
North    25.112360
East     24.167300
Name: Quantity_Sold, dtype: float64
Average Quantity_Sold by Product_Category
Product_Category
Furniture      25.880769
Clothing       25.828358
Food           24.814159
Electronics    24.780488
Name: Quantity_Sold, dtype: float64
Average Quantity_Sold by Customer_Type
Customer_Type
New          26.158730
Returning    24.538306
Name: Quantity_Sold, dtype: float64
Average Quantity_Sold by Payment_Method
Payment_Method
Cash             26.255591
Bank Transfer    25.426901
Credit Card      24.466667
Name: Quantity_Sold, dtype: float64
Average Quantity_Sold by Sales_Channel
Sales_Channel
Online    25.823770
Retail    24.908203
Name: Quantity_Sold, dtype: float64
Average Quantity_Sold by Region_and_Sales_Rep
Region_and_Sales_Rep
South-Eve        29.372093
North-David      29.107143
West-Bob         28.611111
West-David       28.607843
```

```
West-David       28.607843
South-Charlie    26.937500
South-Alice      26.933333
East-Charlie     26.864865
South-David      26.767857
West-Charlie     26.155556
West-Eve         26.042553
Name: Quantity_Sold, dtype: float64
```

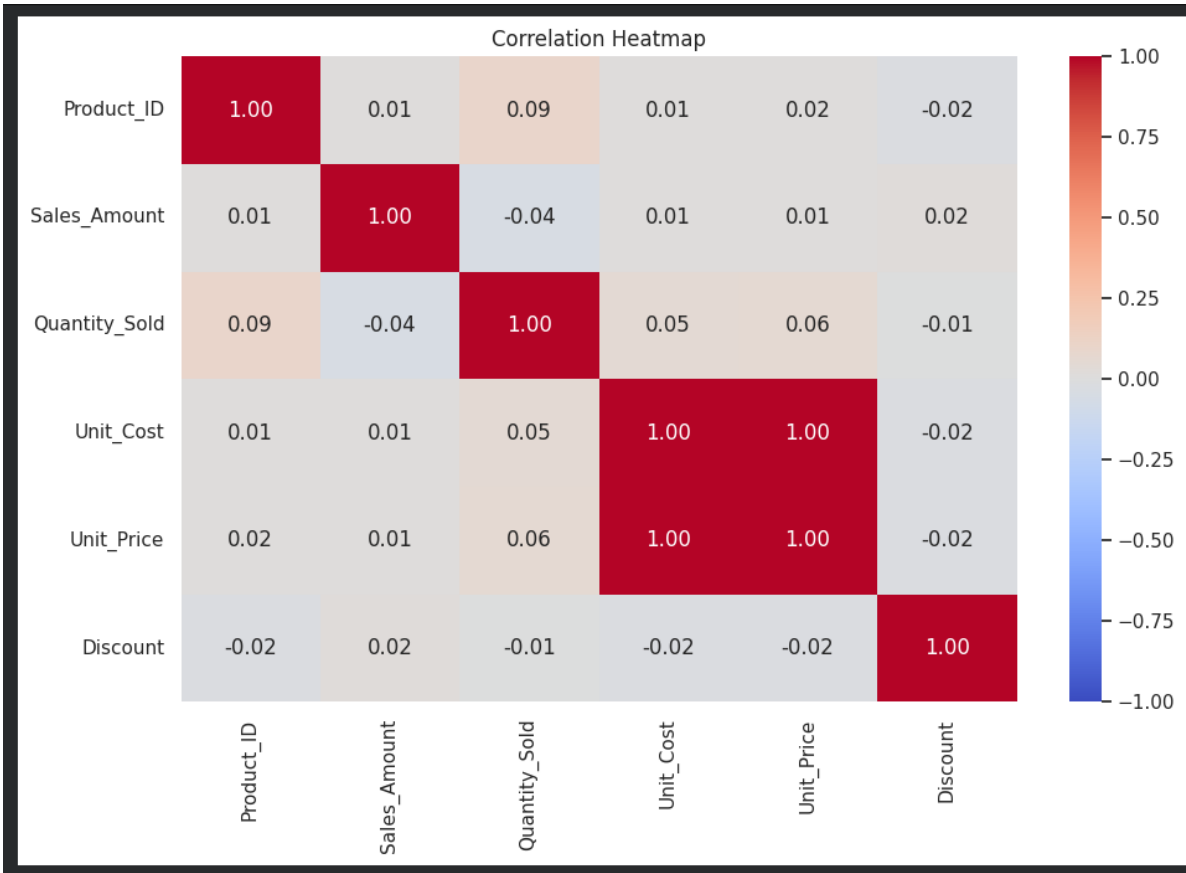
Figure: Outlier analysis

The IQR method found zero outliers in every numerical column. For Sales_Amount the acceptable range is -4,885.42 to 14,943.17, and the actual values lie between 100.12 and 9,989.04, well inside the bounds. This means revenue is not distorted by unusually large or small transactions.

Correlation matrix and heatmap

```
Correlation Matrix:
              Product_ID  Sales_Amount  Quantity_Sold  Unit_Cost  Unit_Price  \
Product_ID          1.00          0.01           0.09       0.01        0.02
Sales_Amount        0.01          1.00          -0.04       0.01        0.01
Quantity_Sold       0.09         -0.04           1.00       0.05        0.06
Unit_Cost           0.01          0.01           0.05       1.00        1.00
Unit_Price          0.02          0.01           0.06       1.00        1.00
Discount           -0.02          0.02          -0.01      -0.02       -0.02

              Discount
Product_ID       -0.02
Sales_Amount      0.02
Quantity_Sold    -0.01
Unit_Cost        -0.02
Unit_Price       -0.02
Discount          1.00
```




```
Strong Positive Relationships:
[('Unit_Cost', 'Unit_Price', np.float64(1.0))]
```

```
Strong Negative Relationships:
None found

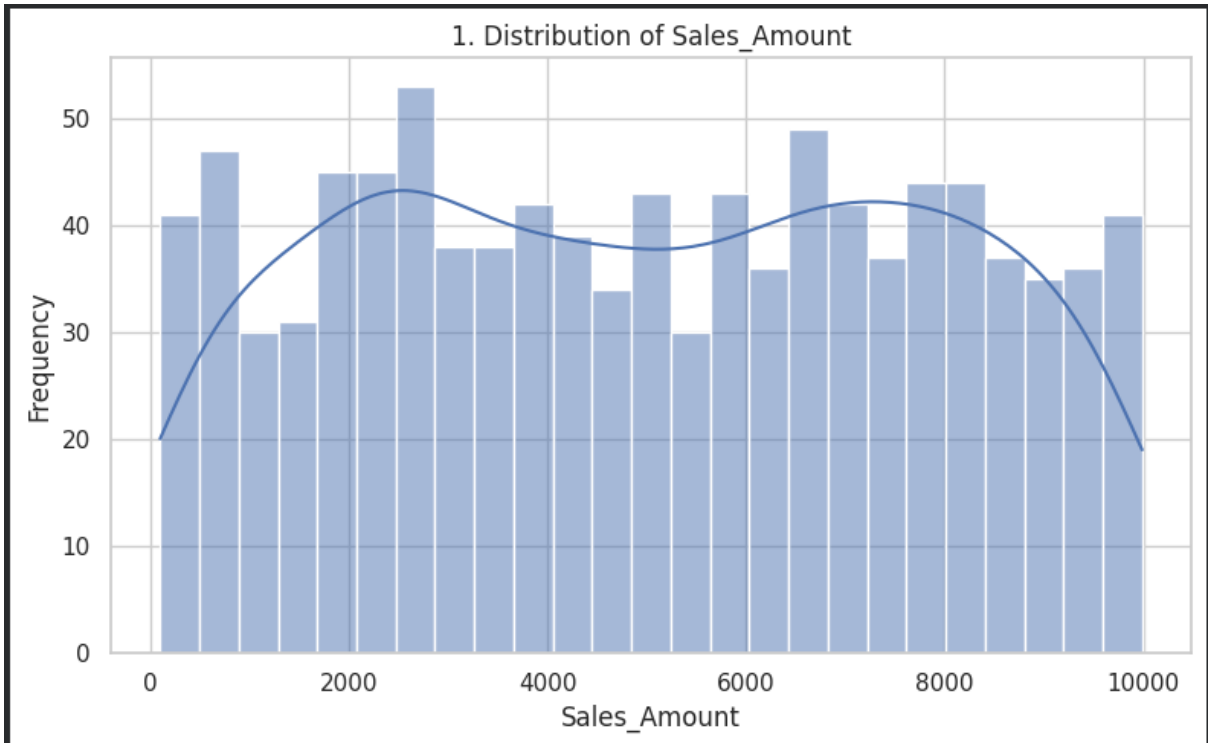
Weak Relationships:
[('Product_ID', 'Sales_Amount', np.float64(0.01)), ('Product_ID', 'Quantity_Sold', np.float64(0.09)), ('Product_ID', 'Unit_Cost', np.float64(0.01)), ('Product_ID', 'Unit_Price', np.

Important Feature Relationships:
Unit_Cost and Unit_Price: 1.00
```

Figure: Correlation matrix and heatmap

The only strong relationship is between Unit_Cost and Unit_Price (about 0.995), which shows that prices closely follow costs. There are no strong negative relationships. All other pairs are weak (below 0.10), for example Sales_Amount and Quantity_Sold (-0.04) and Sales_Amount and Discount (0.02). This means that quantity, price and discount do not explain changes in Sales_Amount in a simple linear way.

Distribution charts: histogram and density plot



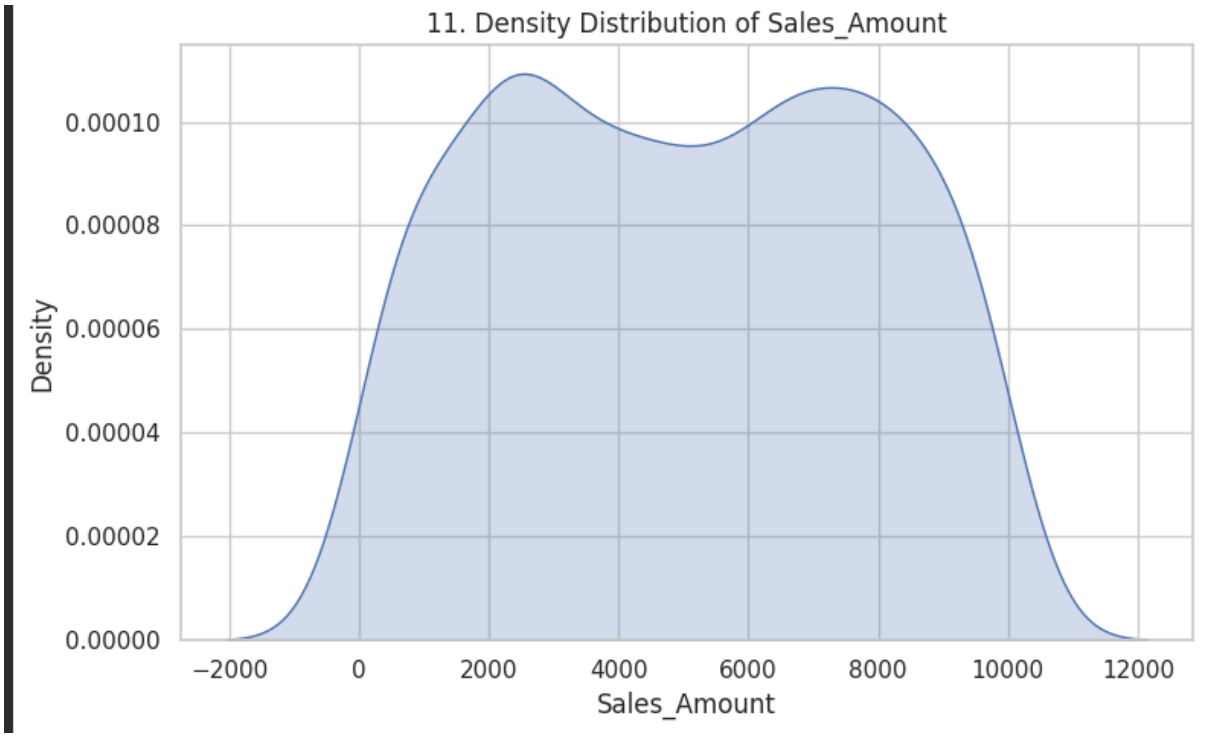
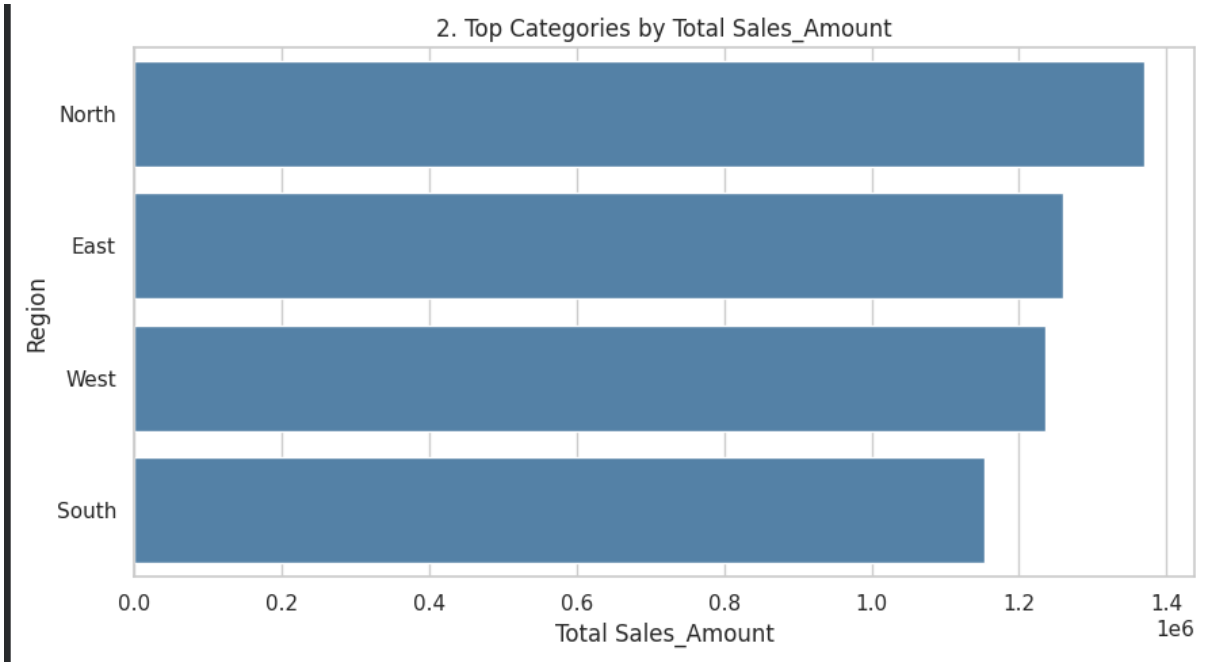
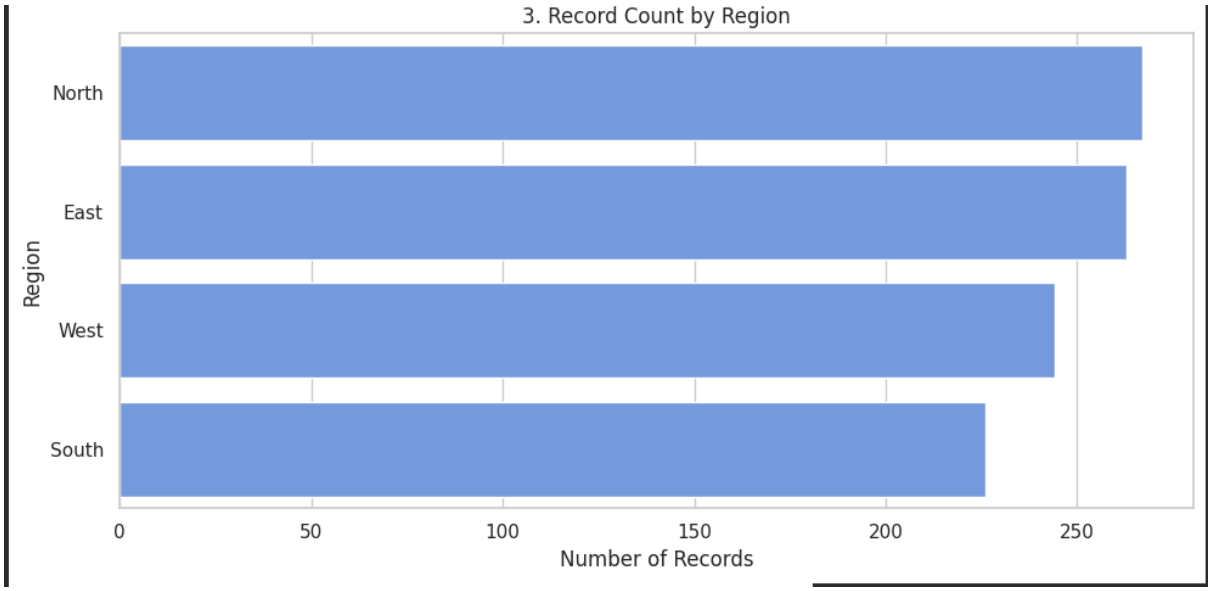


Figure: Distribution charts: histogram and density plot

Both charts show that transaction values are spread fairly evenly between about 100 and 10,000, with no long tail and no dominant peak. This matches the nearly equal mean and median, and shows that orders of many different sizes occur with similar frequency.

Category and regional charts: bar, count and pie charts





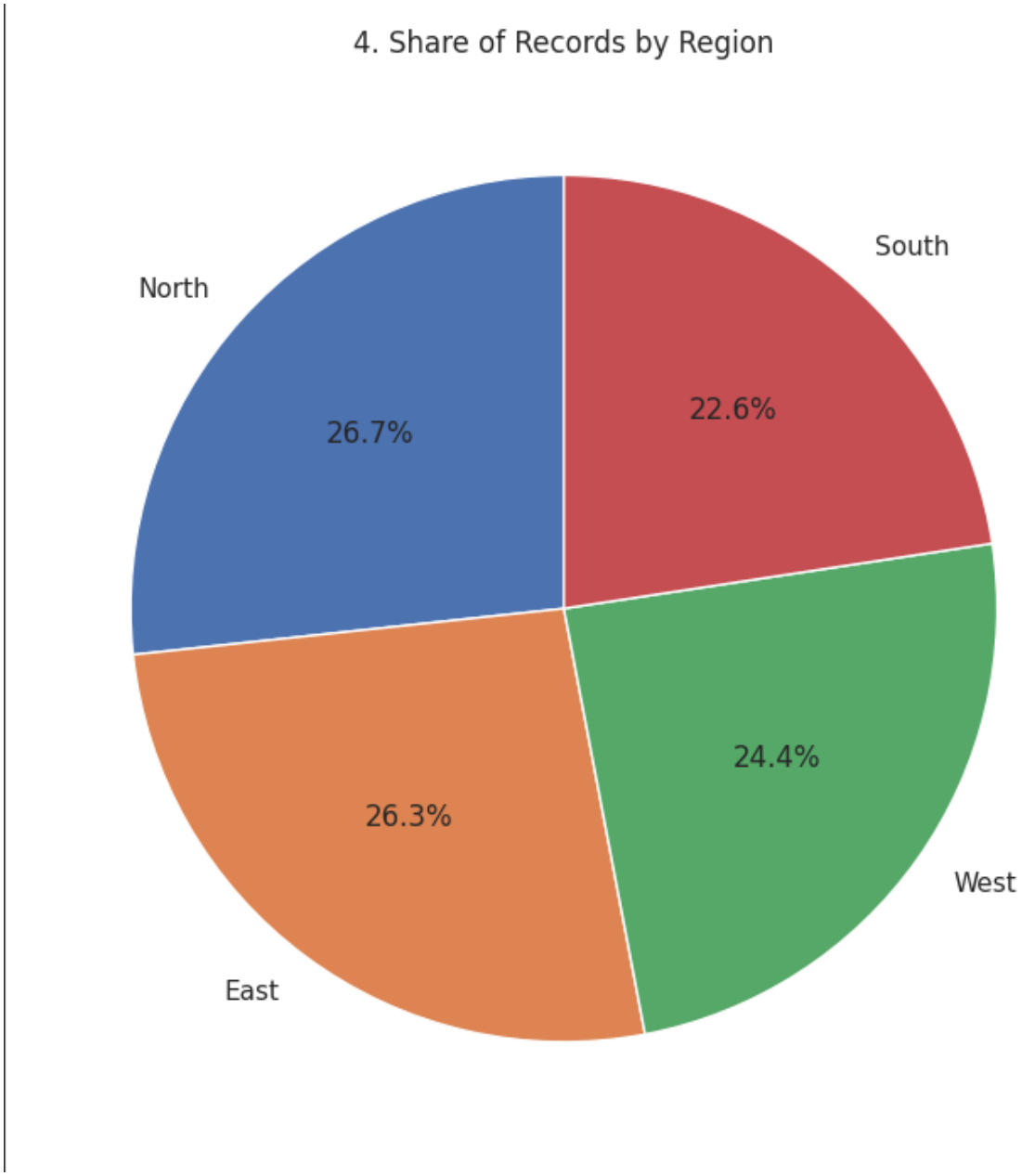


Figure: Category and regional charts: bar, count and pie charts

North generates the highest total sales (1,369,612.51, 27.3 percent of revenue), followed by East (1,259,792.93), West (1,235,608.93) and South (1,154,250.86). The count plot and pie chart show that South also has the fewest transactions (226), which suggests its lower total comes from fewer sales rather than smaller sales.

Spread charts: box plot and violin plot

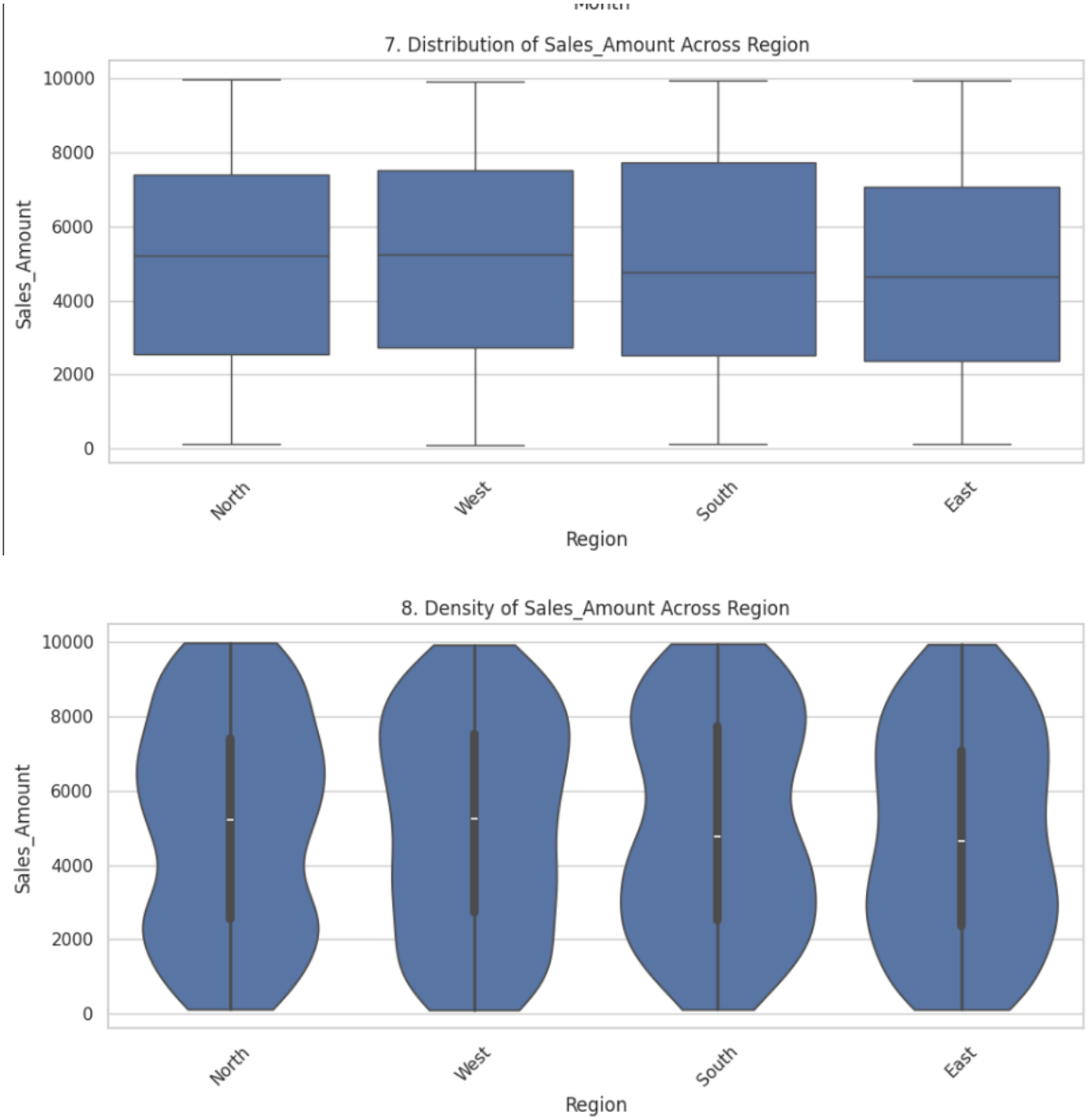
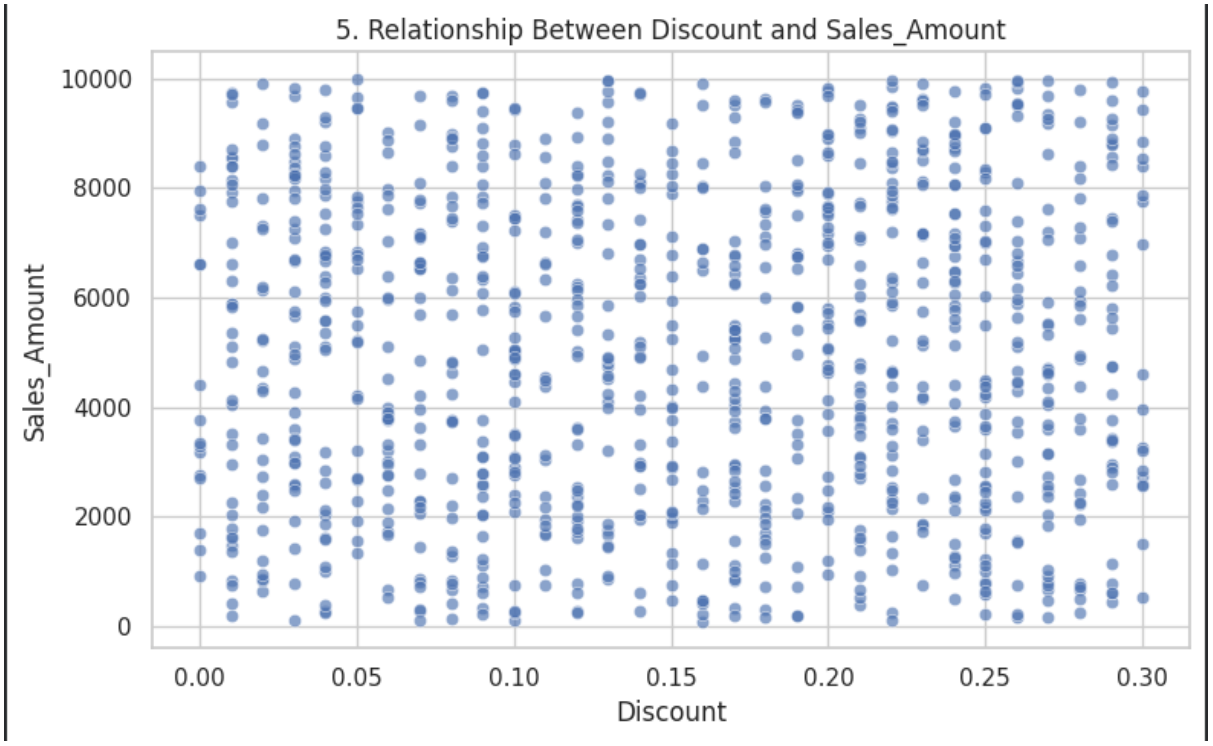
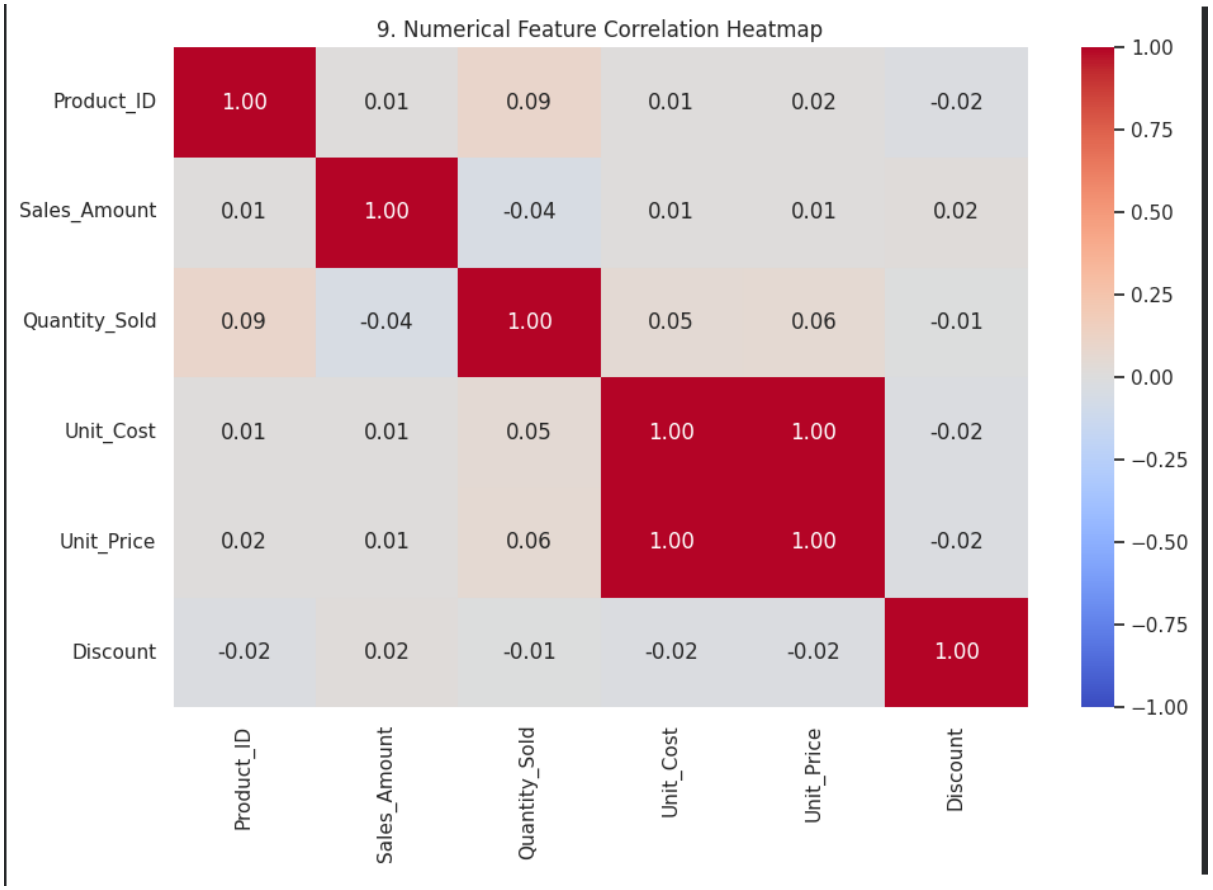


Figure: Spread charts: box plot and violin plot

All four regions have similar medians and similarly wide spreads, and no points fall outside the whiskers, which agrees with the zero-outlier result. The violin shapes are broad and flat, showing that no region depends on a narrow band of order sizes.

Relationship charts: scatter plot, heatmap and pair plot





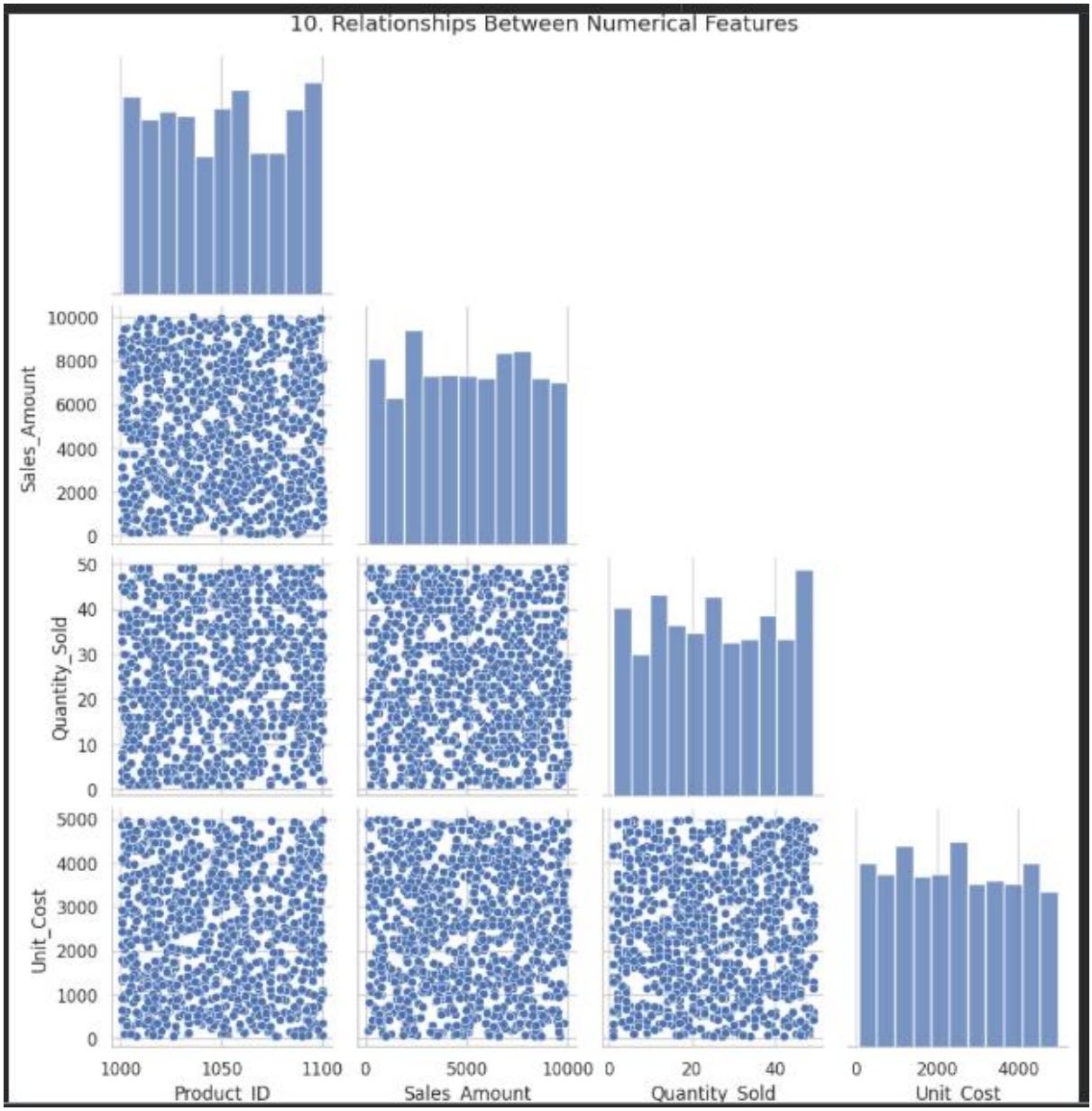


Figure: Relationship charts: scatter plot, heatmap and pair plot

The scatter plot of Discount against Sales_Amount shows a shapeless cloud of points, matching the correlation of only 0.02. The pair plot shows no visible pattern between the selected numerical features. Together with the heatmap, these charts confirm that discounts do not noticeably increase sales.

Time trend charts: line chart and area chart

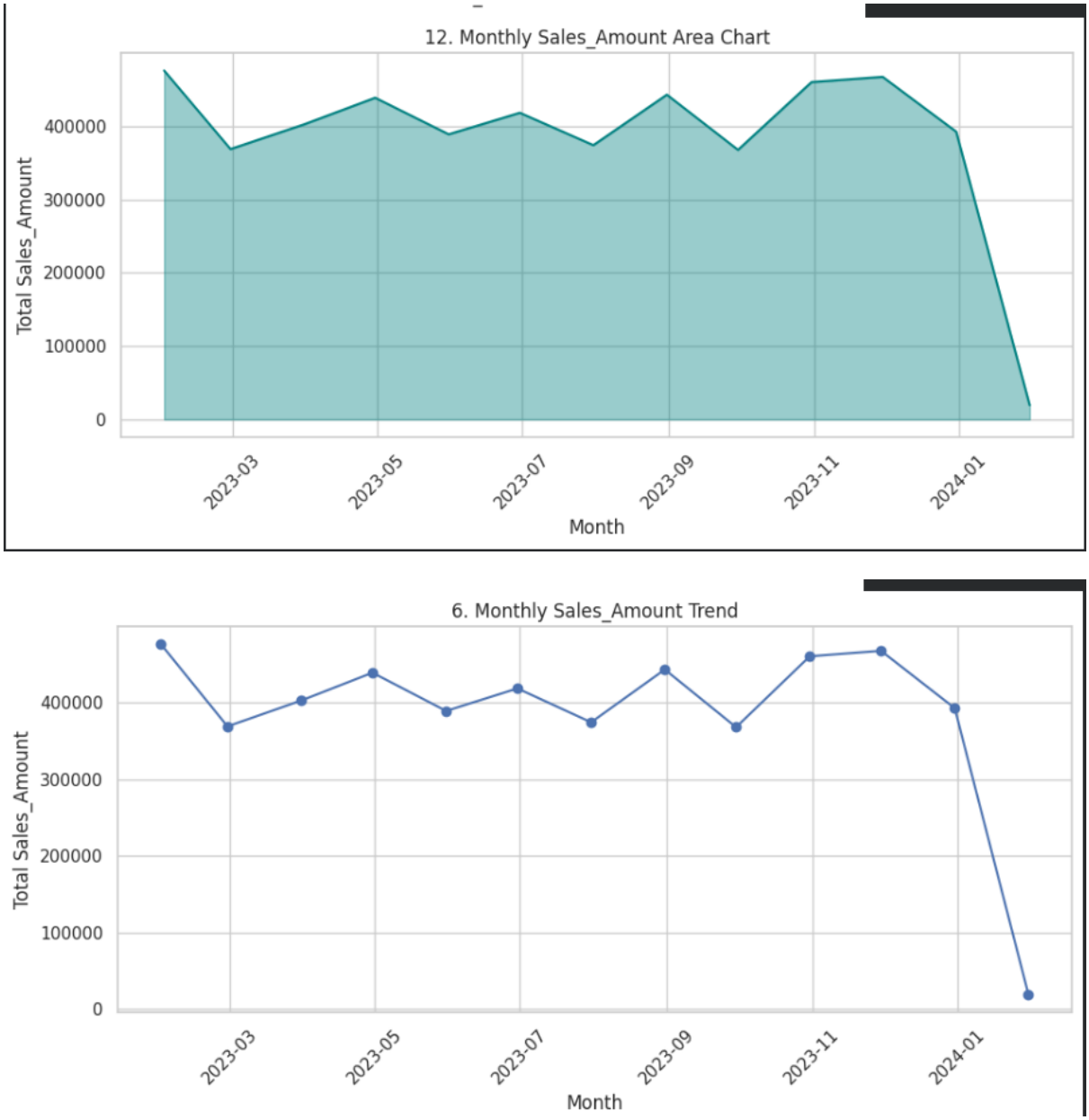


Figure: Time trend charts: line chart and area chart

Monthly sales are highest in January 2023 (476,092.36), November 2023 (467,482.90) and October 2023 (460,378.78), and lowest in September 2023 (367,837.60) and February 2023 (368,919.36). The peak is about 29 percent above the lowest full month, which shows a clear seasonal pattern. The last point (January 2024, 19,328.01) is incomplete data and should not be read as a real drop.

Business performance summary

```
1. Dataset overview
Rows and columns: (1000, 14)
Date range and missing values:
Product_ID              0
Sale_Date               0
Sales_Rep               0
Region                  0
Sales_Amount            0
Quantity_Sold           0
Product_Category        0
Unit_Cost               0
Unit_Price              0
Customer_Type           0
Discount                0
Payment_Method          0
Sales_Channel           0
Region_and_Sales_Rep    0
dtype: int64

2. Numerical summary
       Product_ID  Sales_Amount  Quantity_Sold  Unit_Cost  Unit_Price  \
count     1000.00       1000.00        1000.00    1000.00     1000.00
mean      1050.13       5019.27          25.36    2475.30     2728.44
std         29.57       2846.79          14.16    1417.87     1419.40
min       1001.00        100.12           1.00      60.28      167.12
25%       1024.00       2550.30          13.00    1238.38     1509.08
50%       1051.00       5019.30          25.00    2467.23     2696.40
75%       1075.00       7507.45          38.00    3702.86     3957.97
max       1100.00       9989.04          49.00    4995.30     5442.15

       Discount
count   1000.00
mean       0.15
std        0.09
min        0.00
25%        0.08
50%        0.15
75%        0.23
max        0.30

3. Category performance

Category: Sales_Rep
Highest sales:
Sales_Rep
David      1141737.36
Bob        1080990.63
```

```
    Eve         970183.99
    Alice       965541.77
...  Charlie     860811.48
    Name: Sales_Amount, dtype: float64
    Lowest sales:
    Sales_Rep
    Charlie     860811.48
    Alice       965541.77
    Eve         970183.99
    Bob        1080990.63
    David      1141737.36
    Name: Sales_Amount, dtype: float64

    Category: Region
    Highest sales:
    Region
    North    1369612.51
    East     1259792.93
    West     1235608.93
    South    1154250.86
    Name: Sales_Amount, dtype: float64
    Lowest sales:
    Region
    South    1154250.86
    West     1235608.93
    East     1259792.93
    North    1369612.51
    Name: Sales_Amount, dtype: float64

    Category: Product_Category
    Highest sales:
    Product_Category
    Clothing       1313474.36
    Furniture      1260517.69
    Electronics    1243499.64
    Food           1201773.54
    Name: Sales_Amount, dtype: float64
    Lowest sales:
    Product_Category
    Food           1201773.54
    Electronics    1243499.64
    Furniture      1260517.69
    Clothing       1313474.36
    Name: Sales_Amount, dtype: float64

    Category: Customer_Type
    Highest sales:
```

```
Customer_Type
Returning    2513006.93
New          2506258.30
Name: Sales_Amount, dtype: float64
Lowest sales:
Customer_Type
New          2506258.30
Returning    2513006.93
Name: Sales_Amount, dtype: float64

Category: Payment_Method
Highest sales:
Payment_Method
Credit Card      1757563.52
Bank Transfer    1717363.17
Cash             1544338.54
Name: Sales_Amount, dtype: float64
Lowest sales:
Payment_Method
Cash             1544338.54
Bank Transfer    1717363.17
Credit Card      1757563.52
Name: Sales_Amount, dtype: float64

Category: Sales_Channel
Highest sales:
Sales_Channel
Retail    2560431.30
Online    2458833.93
Name: Sales_Amount, dtype: float64
Lowest sales:
Sales_Channel
Online    2458833.93
Retail    2560431.30
Name: Sales_Amount, dtype: float64

Category: Region_and_Sales_Rep
Highest sales:
Region_and_Sales_Rep
South-David      311416.99
East-Bob         309876.11
North-Eve        304172.20
North-Charlie    299789.01
West-Bob         286449.28
Name: Sales_Amount, dtype: float64
Lowest sales:
Region_and_Sales_Rep
```

```
South-Charlie    156140.68
East-Charlie     165172.94
South-Eve        196078.22
West-Alice       215690.54
West-Eve         230592.67
Name: Sales_Amount, dtype: float64

4. Sales and profit indicators
Total sales: 5019265.23
Average sales: 5019.27
Median sales: 5019.3
Highest transaction: 9989.04
Lowest transaction: 100.12
Total quantity: 25355
Average quantity: 25.36
Average discount: 0.15

5. Correlation analysis
Unit_Cost      Unit_Price       0.995
Product_ID     Quantity_Sold    0.094
Quantity_Sold  Unit_Price       0.057
               Unit_Cost        0.054
Sales_Amount   Quantity_Sold   -0.042
               Discount         0.023
Product_ID     Discount        -0.022
Unit_Cost      Discount        -0.018
Unit_Price     Discount        -0.017
Product_ID     Unit_Price       0.015
               Unit_Cost        0.013
Sales_Amount   Unit_Price       0.012
Product_ID     Sales_Amount     0.010
Sales_Amount   Unit_Cost        0.010
Quantity_Sold  Discount        -0.008
dtype: float64

6. Outlier analysis
Product_ID Potential outliers: 0
Sales_Amount Potential outliers: 0
Quantity_Sold Potential outliers: 0
Unit_Cost Potential outliers: 0
Unit_Price Potential outliers: 0
Discount Potential outliers: 0

7. Date trends
Monthly sales:
Sale_Date
2023-01-31    476092.36
```

```
2023-02-28    368919.36
2023-03-31    402638.77
2023-04-30    438992.61
2023-05-31    389078.76
2023-06-30    418458.34
2023-07-31    374242.88
2023-08-31    443171.28
2023-09-30    367837.60
2023-10-31    460378.78
2023-11-30    467482.90
2023-12-31    392643.58
2024-01-31     19328.01
Freq: ME, Name: Sales_Amount, dtype: float64
Best month: 2023-01-31 00:00:00 476092.36
Lowest month: 2024-01-31 00:00:00 19328.01

8. Customer analysis
Unique customers: 2
Top customers by sales:
Customer_Type
Returning    2513006.93
New          2506258.30
Name: Sales_Amount, dtype: float64
```

Figure: Business performance summary

Total revenue is 5,019,265.23 from 25,355 units sold. By category, Clothing leads (1,313,474.36) and Food is lowest (1,201,773.54), although Food has the highest average sale (5,317.58). By sales representative, David leads (1,141,737.36) and Charlie is lowest (860,811.48). Returning customers (2,513,006.93) and New customers (2,506,258.30) contribute almost equal revenue, and Retail (2,560,431.30) slightly outperforms Online (2,458,833.93).

7. Challenges Faced

Date column was stored as text. The Sale_Date column was read from the CSV as plain text, so it could not be used directly for monthly trends. It had to be converted with pd.to_datetime() before the line chart, area chart and monthly sales calculations would work.

Identifier treated as a number. Product_ID is stored as an integer, so Pandas included it in the numerical statistics and correlation matrix, even though it is only a label. Its weak correlations (for example 0.09 with Quantity_Sold) had to be ignored when interpreting results.

No profit column. Several standard business questions, such as profit margin and profit by category, could not be answered directly. The analysis had to focus on revenue, quantity and discount, and unit price minus unit cost was used only as a rough indicator of margin.

No customer identifier. Because there is no customer ID, customer concentration could not be measured. Only the New versus Returning split was available.

Unexpected relationships. Sales_Amount showed almost no correlation with quantity or unit price, which is unusual for sales data. This required careful wording of the insights, so conclusions were based on what the data actually shows and not on what would normally be expected.

Incomplete final month. January 2024 contained only a small amount of data, so it had to be excluded when choosing the best and worst months.

Making every chart meaningful. The task requires at least 12 visualizations and warns against charts created only to meet the count. Choosing variables so that each chart explained a real pattern took extra thought.

Working across two environments. The code was run in both Google Colab and Jupyter Notebook, so file paths for the dataset and the cleaned CSV had to be kept consistent to avoid errors when switching between them.

8. Conclusion

This task covered the complete EDA workflow on a sales dataset of 1,000 transactions. After cleaning, the data had no missing values, duplicates or negative values, and no statistical outliers. Total revenue was 5,019,265.23, and the almost identical mean and median (about 5,019) show that revenue is spread evenly rather than driven by a few large orders.

The analysis showed that differences between segments come mainly from the number of transactions rather than the size of each sale. Clothing and the North region lead in sales, while Food and the South region trail because of fewer orders, even though Food has the highest average sale. Discounts averaged 15 percent but showed almost no relationship with sales or quantity. Monthly sales follow a seasonal pattern, with peaks in January, October and November and dips in February and September.

Based on these findings, the main recommendations are: grow the number of Food orders, increase sales activity in the South and basket size in the East, test lower discounts before continuing broad discounting, build retention programmes because returning customers generate about half of revenue, and plan inventory around the seasonal peaks. The analysis also showed that the dataset would benefit from profit and customer ID fields, and that Sales_Amount should be validated against quantity and price.

Overall, this task built practical skills in cleaning data, analysing features, interpreting correlations, creating meaningful visualizations and turning evidence into business decisions, which provides a solid base for feature engineering and machine learning in later tasks.

GitHub Repository: https://github.com/ate-eb/Data-Science-Devixo-Tasks.git